

Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Automation and Applied Informatics

Comparison of Reinforcement Learning and Model Predictive Control for a Drone Equipped with a Manipulator

Scientific Students' Association Report

Author:

Tamás Epres

Advisor:

Bence Szinyéri, (BME AUT)
Péter Antal, PhD (HUN-REN SZTAKI)

2026

Contents

Kivonat **3**

Abstract **4**

1 Introduction **1**

2 System Modeling **4**

2.1 Quadcopter Model 4

2.2 Spiral Manipulator 5

2.3 Task Description 6

3 Reinforcement Learning **8**

3.1 Reinforcement Learning Formulation 8

3.1.1 Observation 10

3.1.2 Action 10

3.1.3 Reward 11

3.1.4 Episode 12

3.1.5 Flat Agent 12

3.2 Curriculum Learning 12

3.2.1 Formulation 12

3.2.2 Training with and without CL 13

3.3 Hierarchical Reinforcement Learning 14

3.3.1 Formulation 14

3.3.2 Low-Level Observation and Action 15

3.3.3 Low-Level Reward 16

3.3.4 Low-Level Episode 16

3.3.5 Low-Level Agent Results 16

3.3.6 High-Level Policy 19

3.3.7 High-Level Agent Results 20

4 Model Predictive Control **21**

4.1 Model Predictive Control Formulation 21

4.1.1 State and Input 22

4.1.2 Prediction Model 23

4.1.3 Optimal Control Problem . . . 23
4.2 System Identification . . . 24
4.2.1 Model Structure . . . 24
4.2.2 Training . . . 25
4.2.3 Validation of the Identification . . . 27

5 Safety Filter **29**
5.1 Formulation . . . 30
5.1.1 Integration into the Hierarchy . . . 30
5.1.2 Safety Filter Problem . . . 31
5.1.3 Constraints . . . 32
5.1.4 Additional Rules . . . 32

6 Comparison **33**
6.1 Evaluation Setup . . . 33
6.2 Results . . . 33
6.3 Rescue of the RL Failures . . . 34

7 Conclusion and Future Work **36**

Acknowledgements **38**

Bibliography **39**

Appendix **42**
A.1 Hyperparameters . . . 42
A.2 Technical Details . . . 42
A.3 Notation . . . 44

Kivonat

Az autonóm légi járművek alkalmazása mára szinte mindennapossá vált. Érdekes és ígéretes kutatási irány annak vizsgálata, hogyan használhatók ezek a járművek olyan feladatokra, amelyekben a drón fizikai kapcsolatba lép a környezetével, és kölcsönhatásba kerül vele.

A dolgozat egy ilyen rendszert vizsgál: a sokoldalú megfogás érdekében egy spirálrobotok által ihletett, egyszerűsített, sok szabadságfokú, síkban mozgó manipulátor van egy kvadkopter aljára rögzítve. A dolgozat célja egy szabályozó tervezése erre a rendszerre. A szabályozó feladata olyan trajektória előállítása, amelynek során a drón egy előre nem ismert fizikai paraméterekkel (mérettel és tömeggel) rendelkező csomagot megfog, majd a lehető leggyorsabban a célpontba szállít. A rendszer vizsgálata MuJoCo-alapú szimulációs környezetben történik.

Egy ilyen sok kontaktust tartalmazó, nagy szabadságfokú rendszerre a hagyományos irányítástervezési módszerek, például a PID- vagy az LQR-szabályozás, nem alkalmasak. A megfogás során fellépő kontaktusok, a lengő csomag és az erősen nemlineáris, csatolt dinamika miatt a rendszernek nehéz pontos fizikai modelljét felírni, a linearizált modellek pedig csak egy munkapont szűk környezetében érvényesek. Ezért a dolgozat két, egymást kiegészítő módszert vizsgál: egy modellfüggetlen, tanulásalapú és egy modellalapú, optimalizáláson alapuló megközelítést.

Az első egy Proximal Policy Optimization (PPO) alapú, hierarchikus megerősítéses tanulási (HRL) architektúra. A megerősítéses tanulás adatalapú módszer: az ágens a környezetével interakcióba lépve, annak válaszai alapján tanul meg egy optimális viselkedést. A PPO egy széles körben alkalmazott, stabilan tanítható, folytonos beavatkozójelekre jól használható algoritmus. A hierarchiában egy magas szintű ágens sebességreferenciát generál, amelyet egy előzetesen betanított, alacsony szintű ágens követ, így a drón stabilizálását nem kell a teljes feladattal együtt megtanulni. A tanítást curriculum learning (CL) segíti, amelynek során a feladat nehézsége és a csomag paramétereinek változatossága fokozatosan nő.

A másik megoldás a modellprediktív szabályozás (MPC), amely az állapotokra és a beavatkozójelekre vonatkozó korlátozásokat explicit módon kezeli, és kiszámítható viselkedést biztosít. Az MPC a magas szintű ágens helyét veszi át, és az alacsony szintű ágensnek ad sebességreferenciát. A MuJoCo-modell közvetlen használata az MPC-ben számításigényesebb lehet, a kontaktusok pedig az optimalizálás numerikus stabilitását is veszélyeztethetnék. Ezért a drón és az alacsony szintű ágens együttes viselkedését szimulációs adatokból egy előrecsatolt, reziduális neurális hálózattal (ResNet) identifikáltuk. Ez az architektúra a rendszeridentifikációban bevált, könnyen tanítható, és gyorsan kiértékelhető, folytonosan differenciálható predikciós modellt ad az MPC számára.

Végül a dolgozat javaslatot tesz egy hibrid módszerre, amely ötvözi a két megközelítés erősségeit. Egy prediktív biztonsági szűrő az identifikált modell segítségével ellenőrzi az RL-ágens beavatkozójeleit, és csak akkor avatkozik be, ha azok veszélyes állapotba vinnék a drónt. A három módszert 500 azonos, véletlenszerűen generált feladaton hasonlítjuk össze. Az RL-ágens a leggyorsabb, de a feladatoknak csak a 93,6%-át teljesíti, és főként a nehéz csomagoknál hibázik. Az MPC a legmegbízhatóbb (99,4%), viszont lassabb és számításigényesebb. A biztonsági szűrő 96,8%-ra növeli az RL-ágens sikerességét, miközben megőrzi a gyorsaságát.

Abstract

The use of autonomous aerial vehicles has become almost commonplace. An interesting and promising research direction is to investigate how these vehicles can be used for tasks in which the drone makes physical contact with its environment and interacts with it.

This thesis studies such a system: to enable versatile grasping, a simplified, planar manipulator with many degrees of freedom, inspired by spiral robots, is attached to the underside of a quadcopter. The goal of the thesis is to design a controller for this system. The task of the controller is to generate a trajectory along which the drone grasps a payload with unknown physical parameters (size and mass) and transports it to a goal position as fast as possible. The system is studied in a MuJoCo-based simulation environment.

For such a contact-rich system with many degrees of freedom, classical control design methods such as PID or LQR are not suitable. Due to the contacts during grasping, the swinging payload, and the strongly nonlinear, coupled dynamics, an accurate physical model of the system is hard to obtain, and linearized models are valid only in a narrow neighborhood of an operating point. Therefore, the thesis investigates two complementary methods: a model-free, learning-based approach and a model-based, optimization-based one.

The first one is a hierarchical reinforcement learning (HRL) architecture based on Proximal Policy Optimization (PPO). Reinforcement learning is a data-driven method: the agent learns an optimal behavior by interacting with its environment, based on the responses of the environment. PPO is a widely used algorithm that trains stably and is well suited for continuous actions. In the hierarchy, a high-level agent generates a velocity reference, which is tracked by a pre-trained low-level agent, so the stabilization of the drone does not have to be learned together with the whole task. Training is supported by curriculum learning (CL), during which the difficulty of the task and the variability of the payload parameters are gradually increased.

The other solution is model predictive control (MPC), which handles the constraints on the states and the inputs explicitly and provides predictable behavior. The MPC takes the place of the high-level agent and provides the velocity reference to the low-level agent. Using the MuJoCo model directly in the MPC could be computationally more demanding, and the contacts could compromise the numerical stability of the optimization. Therefore, the combined behavior of the drone and the low-level agent was identified from simulation data with a feedforward residual neural network (ResNet). This architecture has proven well suited for system identification, is easy to train, and provides a continuously differentiable prediction model for the MPC that is fast to evaluate.

Finally, the thesis proposes a hybrid method that combines the strengths of the two approaches. A predictive safety filter checks the commands of the RL agent with the identified model, and it intervenes only if they would drive the drone into an unsafe state. The three methods are compared on the same 500 randomly generated tasks. The RL agent is the fastest, but it completes only 93.6 % of the tasks, failing mainly with heavy payloads. The MPC is the most reliable (99.4 %), but it is slower and computationally more demanding. The safety filter raises the success rate of the RL agent to 96.8 %, while it preserves its speed.

Chapter 1

Introduction

Unmanned aerial vehicles (UAVs) are being used for increasingly complex tasks. Today, their use is no longer limited to passive tasks that require no physical contact with the environment, such as surveillance, video recording, or security and patrol missions. UAVs are now also deployed in tasks where the drone interacts physically with its surroundings. Examples include package delivery from a moving platform [3], the retrieval of flying vehicles [30], pipeline crack sealing [16], and even the collection of endangered plants from dangerous cliffs [14].

This thesis is motivated by the ever faster and steadily growing demand for transportation. Supply chains require increasingly fast and inexpensive delivery, while safe transport and the protection of human life and property remain of paramount importance. An aerial manipulation system of this kind can offer a solution, as the drones appearing on the market are becoming cheaper and faster. Drone delivery is no longer only a research topic: large companies are already introducing it, as shown in Figure 1.1. Amazon plans to extend its package delivery services with the drones of its Prime Air service, and the drones of Wing deliver packages, e.g., of Walmart, which they pick up with a hook on a tether: the drone hovers above the package, lowers the hook, and winches the package up to itself. DJI also offers a dedicated delivery drone, the FlyCart 100, which can carry payloads of up to 85 kg and lowers them with a winch system[1]. In such services, the drone grasps and carries the package autonomously, which is the kind of transportation task that this thesis addresses with a manipulator.

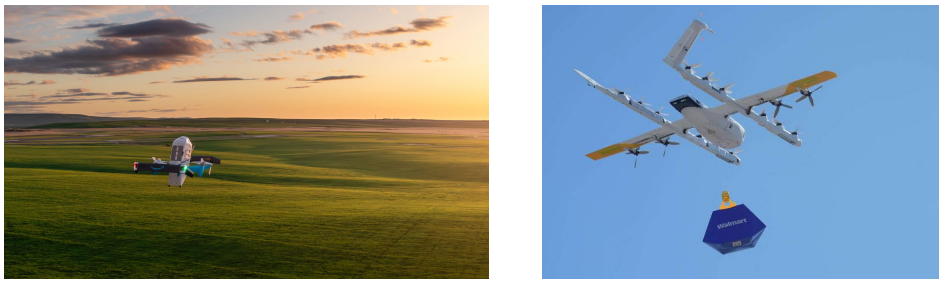


Figure 1.1: Delivery drones: a drone of Amazon Prime Air (left)[2] and a drone of Wing, which lifts the package with a hook on a tether (right)[3].

Despite these advances, several challenges remain before such systems can operate fully autonomously. First, the payload has to be grasped autonomously, whereas current solutions typically rely on packages that are prepared for the specific gripping mechanism of the drone. Second, payloads of various shapes and sizes should be transported, not only standardized packages. Third,

[1]Source: https://www.dji.com/flycart-100

[2]Source: https://www.aboutamazon.com/news/transportation/amazon-prime-air-drone-delivery-expansion

[3]Source: https://www.houstonchronicle.com/business/article/wing-walmart-drone-delivery-21294700.php, see also https://wing.com/

the whole task, from the approach and the grasp to the delivery of a payload whose physical parameters are not known in advance, requires fully autonomous motion planning and control. Accordingly, current research focuses both on the design of manipulators for aerial grasping and on the planning and control of the resulting drone–manipulator systems.

To interact physically with its surroundings, the drone is equipped with some kind of manipulator, which can range from a simple hook [3], through a multi-degree-of-freedom robotic arm [16], to a deformable, so-called soft robotic manipulator containing compliant segments [9]. Each of these solutions has its limitations. A hook can only grasp payloads that are equipped with a suitable handle or loop. A robotic arm is heavy and requires many actuators, which reduces the flight time and the payload capacity of the drone. A soft manipulator, in turn, is hard to control precisely, due to its many degrees of freedom and its compliance. To overcome these limitations, a manipulator inspired by spiral robots [32] was found to be the most suitable for this thesis. It can wrap around objects of various shapes and sizes, it is actuated by only two cables, and since it consists of rigid segments, its motion is more predictable than that of a soft manipulator. The manipulator is mounted on the underside of the drone. Figure 1.2 shows it in its relaxed, straight state and wrapped around a payload during transport. The drone–manipulator system is described in detail in the *System Modeling* chapter.

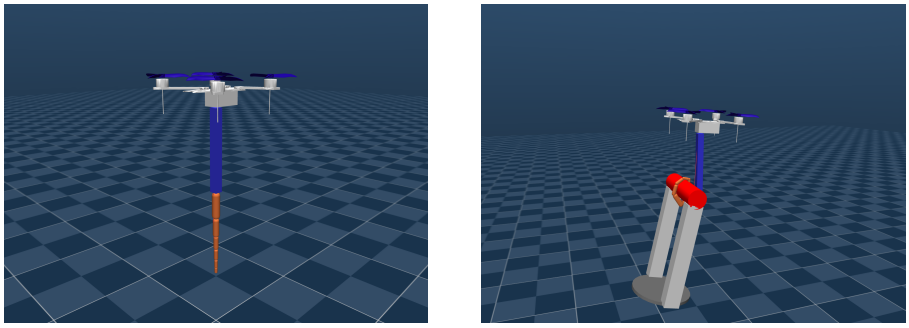


Figure 1.2: The studied drone–manipulator system in the MuJoCo simulation: the spiral-robot-inspired manipulator in its relaxed state without payload (left), and wrapped around the cylinder of the payload while carrying it (right).

For the drone–manipulator system considered in this thesis, no control algorithm has yet been developed that solves such a grasping and transportation task: the work that introduced the spiral robot [32] also mounts it on a drone, but it uses only a simple control approach and does not address the control design of the drone–manipulator system. The problem is challenging for several reasons. The system has many degrees of freedom, its dynamics are strongly nonlinear and coupled, they change considerably when the payload, whose mass and size are not known in advance, is grasped, and the grasping itself involves many contacts. For such systems, classical control design methods, e.g., PID or LQR control, are not suitable, since they rely on a simple or linearized model, which is valid only in a narrow neighborhood of an operating point.

In recent years, learning-based (data-driven) control algorithms have become increasingly widespread, and their application has also extended to drone-mounted manipulator tasks [33]. These methods learn the control policy from data collected either on the real system or in simulation. They are generally robust to modeling errors and computationally efficient at inference time. On the other hand, their training is not trivial and is often a lengthy process, and safe control is difficult to guarantee. As an efficient learning-based control method, reinforcement learning (RL) has been implemented in this thesis. To facilitate training, hierarchical RL (HRL) [12] was applied in combination with curriculum learning (CL) [17]. The method is presented in detail in the *Reinforcement Learning* chapter.

Model predictive control (MPC) [20] has also been applied for real-time trajectory planning and high-performance control of aerial transportation systems [3]. In this framework, a model of the system is available, based on which an optimization problem can be formulated; solving it yields a suitable sequence of control inputs. This control strategy also has its advantages and disadvantages.

Its main advantage is that it can operate even in situations that the drone has not encountered before, while satisfying constraints arising, e.g., from the system dynamics, obstacle avoidance, or actuator limitations. Its disadvantages are that it becomes computationally demanding for higher-dimensional problems, and that it is sensitive to modeling errors. Such an MPC method has been applied in this thesis and is presented in the *Model Predictive Control* chapter, which also describes the system identification procedure used.

Finally, this thesis proposes a solution that combines the two methods so that each compensates for the shortcomings of the other. The two methods are used together in numerous applications [21]; this thesis investigates how MPC can be used to improve the safety of the RL controller. To this end, a predictive safety filter similar to the one in [31] is presented. The filter is an MPC that supervises the control inputs proposed by the RL agent and overrides them whenever they would drive the system into an unsafe state. The method is described in detail in the *Safety Filter* chapter. The three approaches are then compared in the *Comparison* chapter.

To conclude, the main research questions of this work are as follows:

1. How can a control method be designed that achieves successful grasping and transportation of a payload with unknown physical parameters with the spiral manipulator, while remaining computationally tractable for real-time operation?
2. How do reinforcement learning and model predictive control compare in performing the transportation task, in terms of success rate, speed, and computational cost?
3. How can reinforcement learning and model predictive control be combined efficiently to achieve high performance, safety, and computational efficiency in payload transportation?

Chapter 2

System Modeling

This chapter presents the quadcopter, the spiral-robot-inspired manipulator mounted on it [32], and the transportation task to be performed.

2.1 Quadcopter Model

The drone used in this work is the Bumblebee quadcopter of SZTAKI's AIMotionLab. Its modeling and control are described in detail in [2], which the following summary is based on.

The motion of the quadcopter is described in two coordinate frames (Figure 2.1): the inertial frame $\mathcal{F}^i$, fixed to the environment, and the body frame $\mathcal{F}^b$, attached to the center of mass of the drone. Let $SE(3)$ and $SO(3)$ denote the configuration manifold of rigid body dynamics and the group of rotation matrices, respectively. The configuration of the drone is given by the pair $(p, \mathbf{R}) \in SE(3)$, where $p \in \mathbb{R}^3$ is its position in $\mathcal{F}^i$, also referred to as the world frame, and $\mathbf{R} \in SO(3)$ is the rotation matrix from $\mathcal{F}^i$ to $\mathcal{F}^b$. The orientation can equivalently be described by the roll, pitch, and yaw angles $\eta = [\phi,\, \theta,\, \psi]^T$, with

$$\mathbf{R}(\phi, \theta, \psi) = \mathrm{Rot}_x(\phi)\,\mathrm{Rot}_y(\theta)\,\mathrm{Rot}_z(\psi). \tag{2.1}$$

Since Euler angles suffer from singularities, the rotation matrix representation is preferred for control design.

Following [2], the quadcopter is modeled as a rigid body with mass m and inertia matrix J, actuated by four propellers (Figure 2.2). Each propeller $i \in \{1, 2, 3, 4\}$ generates a thrust force and a drag torque proportional to the square of its angular velocity Ω_i:

$$f_i = c\,\Omega_i^2, \qquad \tau_{z,i} = b\,\Omega_i^2, \tag{2.2}$$

where c and b are the thrust and drag coefficients. The collective thrust f and the torques in the body frame are

$$\begin{aligned} f &= f_1 + f_2 + f_3 + f_4, \\ \tau_x &= l_x(f_3 + f_4 - f_1 - f_2), \\ \tau_y &= l_y(f_2 + f_3 - f_1 - f_4), \\ \tau_z &= b(\Omega_1^2 + \Omega_3^2 - \Omega_2^2 - \Omega_4^2), \end{aligned} \tag{2.3}$$

where l_x and l_y are the distances of the propellers from the body axes. The dynamics follow the Newton–Euler equations

$$\begin{aligned} m\ddot{p} &= -mge_3 + f\mathbf{R}e_3, \\ J\dot{\omega} &= \tau - \omega \times J\omega, \\ \dot{\mathbf{R}} &= \mathbf{R}\hat{\omega}, \end{aligned} \tag{2.4}$$

where $e_3 = [0, 0, 1]^T$, $\omega \in \mathbb{R}^3$ is the angular velocity in $\mathcal{F}^b$, and $\hat{\omega}$ is its skew-symmetric matrix. The states p, $\mathbf{R}$, and ω, as well as the inputs f and τ, are time-dependent; for brevity, this dependence is not denoted explicitly. Note that ω is not the time derivative of the Euler angles: the two are related by $\dot{\eta} = W(\eta)\,\omega$, where $W(\eta)$ is the Euler angle rate matrix [6]. The inputs of the model are the collective thrust f and the torque vector $\tau = [\tau_x, \tau_y, \tau_z]^T$.

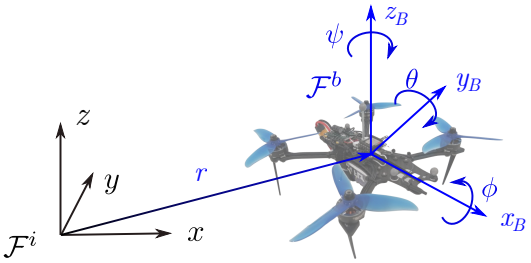


Figure 2.1: Inertial and body frames of the quadcopter, reproduced from [2, Fig. 3].

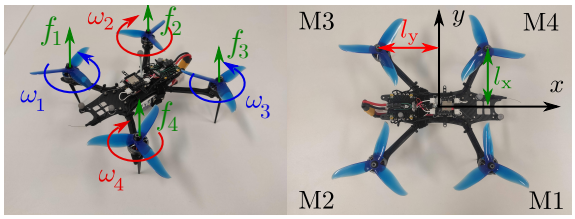


Figure 2.2: Propeller angular velocities and thrusts of the quadcopter, reproduced from [2].

The parameters of the model, measured in [2], are summarized in Table 2.1. The inertia matrix was computed from a detailed CAD model of the drone:

$$J = \begin{bmatrix} 1590.5 & -3.14 & -0.135 \\ -3.14 & 1481.3 & -49.18 \\ -0.135 & -49.18 & 2768.4 \end{bmatrix} \text{kg mm}^2. \tag{2.5}$$

Symbol	Description	Value/Unit
m	Mass of the quadcopter	0.605 kg
J	Inertia matrix	see (2.5) kg mm^2
l_x	Propeller distance from the y_B axis	0.091 m
l_y	Propeller distance from the x_B axis	0.087 m
c	Thrust coefficient	9.39×10^{-7} N s^2/rad^2
b	Drag coefficient	5.59×10^{-7} N m s^2/rad^2
g	Gravitational acceleration	9.81 m/s^2

Table 2.1: Parameters of the quadcopter model.

2.2 Spiral Manipulator

The spiral robot is named after a pattern that occurs frequently in nature, the logarithmic spiral, which can be observed, for example, in the trunk of an elephant or the arm of an octopus. Similarly to these biological examples, the robot can grasp objects of various shapes and masses, such as a tennis ball, an apple, or a pair of headphones [32].

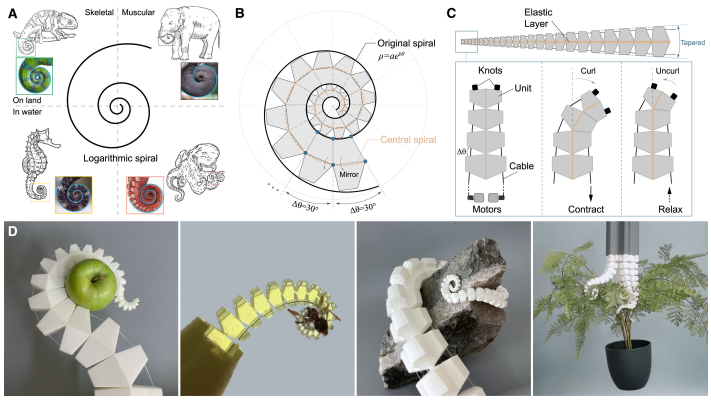


Figure 2.3: Applications and scalability of the spiral robot, reproduced from [32, Fig. 1]. (A) Logarithmic spirals in animal body parts (fitted spirals in blue). (B) Design: the discretized spiral is mirrored to form the robot units. (C) Elastic central layer and motor-driven cables that make the robot curl and uncurl. (D) 3D-printed robots grasping an apple, an ant, a rock, and a plant.

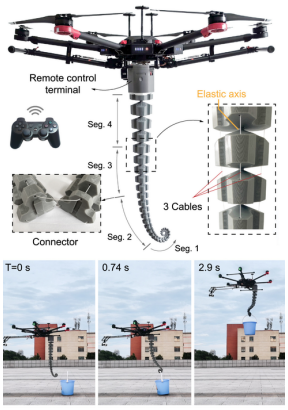


Figure 2.4: The spiral robot mounted on a drone picks up a bucket, reproduced from [32, Fig. 5(E), (F)].

As shown in Figure 2.3(B) and (C), the spiral robot is not a soft manipulator. Instead, it consists of a series of rigid, 3D-printed segments that discretize the logarithmic spiral. The segments are connected by an elastic layer, which pulls the robot back to a straight configuration in its rest state. The robot is actuated by cables, which are routed along the edges of the segments and fixed to the last segment. When one of the cables is pulled, the robot curls in the corresponding direction, as illustrated in Figure 2.3(C).

This thesis can be regarded as an extension of [32]. Although [32] also mounts the spiral robot on a drone (Figure 2.4), it focuses on the fabrication, load-bearing capacity, contact sensing, and scalability of the robot, without addressing the modeling or a rigorous control design of the drone–manipulator system.

In this thesis, a simplified version of the spiral robot is modeled in the MuJoCo simulation environment [27], as shown in Figure 1.2. The arm consists of seven segments, and with the body frame $\mathcal{F}^b$ of Figure 2.1, it moves only in the x_B–z_B plane.

The MuJoCo simulation environment provides an accurate model of such a drone–manipulator system, which can also be used directly within MPC. However, this approach has two potential drawbacks. First, the contact dynamics between the manipulator and the payload may cause numerical instability in the optimization. Second, for systems with many degrees of freedom, the real-time evaluation of the model together with its derivatives may not be feasible, since the derivatives required by the optimizer are obtained by finite differencing (FD), as noted in [15]. The situation is further complicated by the fact that the drone carries a payload with unknown physical parameters, which changes the dynamics of the system. To address these issues, a data-driven identification procedure is presented in detail in the *Model Predictive Control* chapter.

2.3 Task Description

In this work, a typical aerial pick-and-place scenario is considered with the introduced manipulator. The drone always starts from the same initial position with a random yaw orientation. A payload with unknown physical parameters is placed at a random position, and a goal position is also chosen randomly. The task of the drone is to pick up the payload and then fly to the goal position while carrying it. A typical setup is shown in Figure 2.5.

The payload is designed to resemble a package with a handle, as shown in Figure 2.6. It consists of a horizontal cylinder, which serves as the handle, and two cuboids, which are attached to the same side of the cylinder at the two ends of its lateral surface and connect it to a disk at the bottom.

Since the cylinder is held above the disk, there is free space around it, so the manipulator can wrap around it completely. Most of the mass of the payload is concentrated in the disk, which hangs below the hook during transport and causes the payload to swing. The radius of the cylinder and the mass of the disk are randomized in each task, as described below.

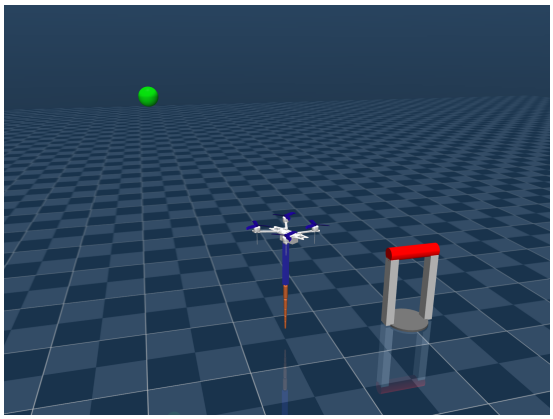


Figure 2.5: A typical scene with a randomly placed payload, a random goal position, and a random initial yaw angle of the drone.

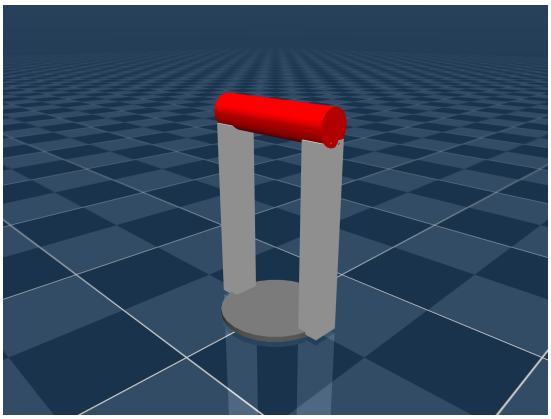


Figure 2.6: Structure of the payload. The robot grasps the red cylinder; its radius and the mass of the disk are not known in advance.

In each task, the radius R_{payload} of the cylinder, the mass m_{payload} of the disk, the height z_{payload} and the horizontal position $(x_{\text{payload}}, y_{\text{payload}})$ of the payload, the horizontal goal position $(x_{\text{goal}}, y_{\text{goal}})$, and the initial yaw angle ψ_0 of the drone are sampled from uniform distributions, whose bounds are summarized in Table 2.2. The height of the goal position is fixed at $z_{\text{goal}} = 1\,\text{m}$.

Parameter	Minimum	Maximum	Unit
Cylinder radius R_{payload}	0.02	0.04	m
Disk mass m_{payload}	0.01	0.25	kg
Payload height z_{payload}	0.45	0.8	m
Payload position x_{payload}, y_{payload}	-1	1	m
Goal position x_{goal}, y_{goal}	-1.5	1.5	m
Initial yaw angle ψ_0	0	2π	rad

Table 2.2: Bounds of the randomized task parameters.

Chapter 3

Reinforcement Learning

This chapter presents a data-driven approach to solving the task, based on reinforcement learning (RL). The RL literature dates back several decades [24], but its major successes came only with the growth of available computational power, and numerous applications have emerged since [23], [11].

RL is a natural choice for the problem described in the *Task Description* section. One of its key properties is that it is data-driven: the agent learns its behavior by interacting with the environment, here the MuJoCo simulation, based on the responses of the environment, so the controller neither evaluates the model nor requires its derivatives online. This is particularly advantageous here, since the contacts of the drone–manipulator system carrying an unknown payload may cause numerical difficulties in model-based optimization. Moreover, since the policy is trained in simulation, techniques such as domain randomization can be used to make it robust to modeling errors, which facilitates the transfer of the learned policy from simulation to the real system [34]. In this thesis, the widely used Proximal Policy Optimization (PPO) algorithm [22] is applied. The first part of the chapter introduces the RL methodology in the context of the thesis.

Despite these advantages, classical RL-based control also has several limitations. First, training is often computationally demanding and, depending on the complexity of the task, may take several hours on a conventional PC. Second, in more complex environments, the agent may get stuck in local optima that do not produce the desired behavior. To overcome these limitations, the second and third parts of this chapter show how curriculum learning (CL) [17] and hierarchical reinforcement learning (HRL) [12] can be applied efficiently.

3.1 Reinforcement Learning Formulation

Throughout this chapter, $\mathrm{Dist}(\mathcal{X})$ denotes the set of probability distributions over a set $\mathcal{X}$. For example, a Gaussian distribution over the actions is an element of $\mathrm{Dist}(\mathcal{A})$, where $\mathcal{A}$ is the set of actions.

The interaction between the agent and its environment is modeled as an episodic Markov decision process (MDP) [24]

$$\mathcal{M} = (\mathcal{S},\, \mathcal{A},\, \mathcal{P},\, r,\, \rho_0,\, \mathcal{S}_f)\,, \tag{3.1}$$

where $\mathcal{S}$ and $\mathcal{A}$ are the state and action spaces, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \mathrm{Dist}(\mathcal{S})$ is the transition function, i.e., $\mathcal{P}(s' \mid s, a)$ is the probability of reaching state s' after taking action a in state s, $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function, $\rho_0 \in \mathrm{Dist}(\mathcal{S})$ is the initial state distribution, and $\mathcal{S}_f \subset \mathcal{S}$ is the set of terminal states.

At each time step t, the environment is in state $S_t \in \mathcal{S}$. The agent, however, does not have access to the full state: the physical parameters of the payload, for instance, are not observed. Such a decision process, in which the state is only partially accessible to the agent, is referred to as a partially observable Markov decision process (POMDP) [10]. A POMDP extends the MDP (3.1) with an observation space $\mathcal{O}$ and an observation function $\mathcal{Z} : \mathcal{S} \to \mathrm{Dist}(\mathcal{O})$, which describes the

observation $O_t \sim \mathcal{Z}(\cdot \mid S_t)$ that the agent receives in state S_t. In this thesis, the observation is a deterministic function of the state, i.e., O_t contains only a part of the information in S_t, as detailed in Section 3.1.1.

The agent selects its actions according to a stochastic policy, which maps an observation to a probability distribution over the actions [21]. In continuous, high-dimensional observation and action spaces, such as those of the transportation task, the policy cannot be represented in tabular form, so it is approximated by a parameterized function with parameters ζ:

$$\pi_\zeta : \mathcal{O} \to \mathrm{Dist}(\mathcal{A}) \tag{3.2}$$

In modern reinforcement learning methods, π_ζ is typically a neural network, which is referred to as deep reinforcement learning [5]. In this thesis, π_ζ is a feedforward neural network, whose output is the mean of a Gaussian distribution over the actions, while its standard deviation is a separately learned parameter. During training, actions are sampled from this distribution, which ensures exploration, whereas during evaluation, the mean is applied as a deterministic action. Feedforward policy networks trained in simulation in this way have already been used successfully for the agile control of quadcopters [11].

The resulting interaction between the agent and the environment is shown in Figure 3.1. At each time step, the agent samples an action from the distribution that its policy assigns to the current observation, $A_t \sim \pi_\zeta(\cdot \mid O_t)$. In response, the environment transitions to a new state $S_{t+1} \sim \mathcal{P}(\cdot \mid S_t, A_t)$, and it returns a scalar reward $R_{t+1} = r(S_t, A_t, S_{t+1})$ and the next observation $O_{t+1} \sim \mathcal{Z}(\cdot \mid S_{t+1})$. The episode starts from an initial state $S_0 \sim \rho_0$ and lasts until a terminal state in $\mathcal{S}_f$ is reached or the time limit is exceeded, at time step T.

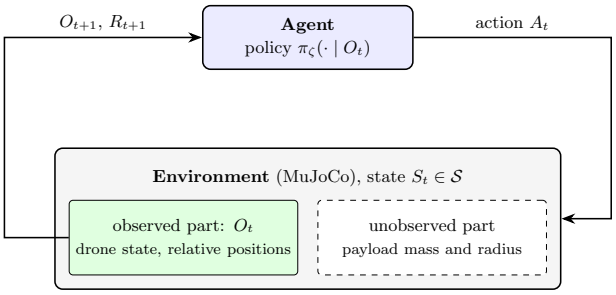


Figure 3.1: Interaction of the agent and the environment. Based on the observation O_t, the agent selects an action A_t with its policy π_ζ, in response to which the environment returns the reward R_{t+1} and the next observation O_{t+1}.

The goal of the agent is to maximize the return, i.e., the discounted cumulative reward collected from time step t until the end of the episode,

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} R_{t'+1}, \tag{3.3}$$

where $\gamma \in [0, 1]$ is the discount factor. Accordingly, training seeks the policy parameters that maximize the expected return from the initial state:

$$\mathcal{J}(\zeta) = \mathbb{E}_{\pi_\zeta}[G_0]\,, \qquad \zeta^\star = \arg\max_\zeta \mathcal{J}(\zeta). \tag{3.4}$$

In this thesis, $\mathcal{J}(\zeta)$ is maximized with Proximal Policy Optimization (PPO) [22], an on-policy actor-critic algorithm that limits the change of the policy in each update by a clipped objective. PPO was chosen for two reasons. First, in drone control applications, it has been shown to outperform other policy gradient methods: in the attitude control of a quadcopter, the PPO agent outperformed Trust Region Policy Optimization (TRPO) and Deep Deterministic Policy Gradient

(DDPG) in every evaluated measure [13]. Second, several well-tested, open-source implementations of PPO are available, such as that of Stable-Baselines3 [19], which is used in this thesis.

All policies are trained with the same setup, with 8 parallel training environments and the same hyperparameters, which are detailed in Appendix A.1.

The following subsections define the observation, the action, the reward, and the episode of the agent that solves the transportation task.

3.1.1 Observation

At time step t, the observation $O_t \in \mathbb{R}^{35}$ is defined as

$$O_t = \left[p_t^T,\ v_t^T,\ \eta_t^T,\ \omega_t^T,\ (w_t - p_t)^T,\ d_{2,t}^T, \ldots, d_{7,t}^T,\ l_{1,t},\ l_{2,t}\right]^T, \tag{3.5}$$

where every component is the value of the corresponding quantity at time step t.

The first four components describe the motion of the drone. The position $p_t \in \mathbb{R}^3$ and the linear velocity $v_t \in \mathbb{R}^3$ are needed to plan the motion toward the targets, while the roll, pitch, and yaw angles $\eta_t = [\phi_t,\ \theta_t,\ \psi_t]^T$ and the angular velocity $\omega_t \in \mathbb{R}^3$ of (2.4) are needed to stabilize the drone and to keep its yaw orientation. The angular velocity is expressed in the body frame, since the motors rotate the drone about its body axes, and on a real drone, it would be measured in this frame by the gyroscope of the onboard inertial measurement unit. All other vector quantities are expressed in the world frame, in which the waypoints and the payload are given. On a real system, p_t, v_t, and η_t could be obtained, e.g., from a motion capture system or from an onboard state estimator.

The relative position $w_t - p_t \in \mathbb{R}^3$ of the waypoint w_t that is active at time step t, defined in the *Reward* subsection, tells the agent where it has to fly in the current phase of the task. Since the waypoints are computed from the task, i.e., from the positions of the payload and the goal, this quantity is available whenever the position of the drone is known. It is given relative to the drone, so that the policy depends only on the distance to the target and not on its absolute location, which helps the policy generalize over the randomly placed payloads and goals.

The relative positions $d_{j,t} = p_{\text{payload},t} - p_{s_j,t} \in \mathbb{R}^3$ of the payload with respect to the j-th segment of the manipulator, $j = 2, \ldots, 7$, describe how the manipulator is placed around the payload. They are needed both to position the hook for the grasp and to recognize whether the payload is held, since the cumulative segment distance of the reward, defined in (3.9), is computed from them. On a real system, the position of the payload could be measured, e.g., with a motion capture system or onboard vision, while the positions of the segments could be estimated from the pose of the drone and the lengths of the tendons with a kinematic model of the manipulator. Since such a model would not account for the deformation of the manipulator under contact, $d_{j,t}$ would be the component of the observation that is the hardest to obtain accurately in practice.

Finally, the normalized lengths $l_{1,t}$ and $l_{2,t}$ of the two tendons describe the current configuration of the manipulator, i.e., how far the hook is closed. On a real system, they could be measured by the encoders of the motors that actuate the tendons.

The observation contains no information about the mass and the radius of the payload, since these are not known in advance and could not be measured directly; the policy therefore has to act robustly over the whole range of payload parameters.

3.1.2 Action

At time step t, the action $A_t = [a_{1,t}, \ldots, a_{6,t}]^T \in [-1, 1]^6$ consists of four normalized motor commands $a_{1,t}, \ldots, a_{4,t}$ and two tendon commands $a_{5,t}$ and $a_{6,t}$. The tendon commands are not learned independently, which follows from the working principle of the manipulator. The spiral robot is designed to adapt passively to the shape of the grasped object: it only has to curl toward the object, and its compliant structure wraps around it [32]. Moreover, the two tendons act antagonistically, i.e., pulling one of them curls the manipulator to one side and pulling the other

one to the opposite side, so a grasp in which both tendons are pulled at the same time does not exist. Therefore, the tendon actuation is described by only two quantities: the side toward which the manipulator curls, and how far it is closed.

The curling direction $\sigma_t \in \{-1, 1\}$ determines toward which side of the drone the hook curls. It is not learned, but determined from the geometry, namely from the side of the payload relative to the second segment of the manipulator, expressed in the body frame of the drone, since the hook always has to curl toward the payload, and learning this discrete choice would only slow down the training. The closing magnitude μ_t is learned from the action of the agent as

$$\mu_t = \frac{a_{5,t} + 1}{2} \in [0, 1], \tag{3.6}$$

where $\mu_t = 0$ corresponds to the relaxed and $\mu_t = 1$ to the fully closed hook, while $a_{6,t}$ is not used. The two tendons are then actuated symmetrically with the commands $\sigma_t \mu_t$ and $-\sigma_t \mu_t$, i.e., one tendon is shortened while the other one is lengthened by the same amount.

Finally, the tendons are actuated only once the payload has come within $R_{\text{payload}} + 0.02\,\text{m}$ of the second segment of the manipulator and the grasp waypoint has been activated; otherwise, they are relaxed.

3.1.3 Reward

The reward consists of a dense shaping term $r_{\text{shape},t+1}$, sparse bonuses $r_{\text{bonus},t+1}$ for completing the subtasks, and a termination penalty $r_{\text{term},t+1}$:

$$R_{t+1} = r_{\text{shape},t+1} + r_{\text{bonus},t+1} + r_{\text{term},t+1}, \tag{3.7}$$

where all terms are evaluated in the state S_{t+1} reached after the action A_t.

Both r_{shape} and r_{bonus} are built around a sequence of waypoints, which divides the task into phases. The waypoints are not learned by the agent: they are computed by the environment at the beginning of each episode from the parameters of the task, and exactly one of them is active at a time; the waypoint active at time step t is denoted by w_t. The first waypoint is the take-off point $[0, 0, 1]^T\,\text{m}$ above the starting position. It is followed by an approach point 0.35 m above the grasp pose, from which the drone descends onto the payload, so that the manipulator does not hit the payload from the side. The grasp pose is computed from the position and the radius of the payload: the drone is placed next to the cylinder, at a lateral distance slightly larger than its radius and at a height at which the tip of the straight manipulator stays above the ground, so that curling the manipulator wraps it around the cylinder. The last waypoint is the goal position.

The active waypoint w_t is considered reached if the horizontal and vertical position errors of the drone satisfy

$$e_{xy,t} = \|p_{xy,t} - w_{xy,t}\| < \varepsilon_{\text{wp}}, \qquad e_{z,t} = |p_{z,t} - w_{z,t}| < \frac{\varepsilon_{\text{wp}}}{5}, \tag{3.8}$$

where $\varepsilon_{\text{wp}} = 0.1\,\text{m}$ is the waypoint tolerance. When a waypoint is reached, the environment activates the next one, except for the grasp pose: there, the next waypoint is activated only once the payload is grasped, as defined below.

To measure how tightly the manipulator is wrapped around the payload, the cumulative segment distance is defined as

$$d_{\text{seg},t} = \left\| \begin{bmatrix} \|d_{2,t}\| - R_{\text{payload}} & \cdots & \|d_{7,t}\| - R_{\text{payload}} \end{bmatrix} \right\|, \tag{3.9}$$

i.e., the norm of the gaps between the surface of the cylinder and the manipulator segments. The payload is considered grasped when $d_{\text{seg},t} < 0.1\,\text{m}$.

The shaping term, applied at every time step, is

$$r_{\text{shape},t+1} = -e_{z,t+1} - 0.1\, e_{xy,t+1} - 0.03 \left\| [\phi_{t+1},\, \theta_{t+1}]^T \right\| - 0.06 \|\omega_{t+1}\| - 0.03\, d_{\text{seg},t+1}, \tag{3.10}$$

where $e_{xy,t+1}$ and $e_{z,t+1}$ are the waypoint errors in (3.8), and $d_{\text{seg},t+1}$ is defined in (3.9). It drives the drone toward the active waypoint, penalizes tilting and rapid rotations, and encourages the manipulator to stay close to the payload.

The sparse bonuses $r_{\text{bonus},t+1}$ depend on the active waypoint and are summarized in Table 3.1. Finally, the termination penalty is $r_{\text{term},t+1} = -100$ if the episode is terminated in S_{t+1}, as described in the *Episode* subsection, and $r_{\text{term},t+1} = 0$ otherwise.

Active waypoint	Condition	Bonus
Take-off	waypoint reached	+20
Approach	waypoint reached	+20
Grasp pose	waypoint reached	+0.1
	$d_{\text{seg},t+1} < 0.1\,\text{m}$ (payload grasped)	+5
Goal	$d_{\text{seg},t+1} < 0.15\,\text{m}$ (payload carried)	+3
	waypoint reached	+3

Table 3.1: Sparse reward bonuses $r_{\text{bonus},t+1}$ depending on the active waypoint.

3.1.4 Episode

At the beginning of each episode, the drone is placed level and at rest at $[0,\, 0,\, 0.5]^T\,\text{m}$, with a random yaw angle, and the payload and the goal are generated randomly. An episode is terminated at time step t if the drone crashes, i.e., $p_{z,t} < 0$, if it flips over, i.e., $|\phi_t| > \pi/2$ or $|\theta_t| > \pi/2$, or if the payload is lost, i.e., its distance from the second segment of the manipulator exceeds 1 m while the goal waypoint is active. Otherwise, the episode is truncated after 10 s.

3.1.5 Flat Agent

In the rest of this thesis, the agent defined by the observation (3.5), the action of Section 3.1.2, the reward (3.7), and the episode of Section 3.1.4 is referred to as the *flat agent*, since it solves the whole transportation task with a single, non-hierarchical policy π_ζ, directly commanding the motors of the drone.

3.2 Curriculum Learning

The main limitation of the flat agent is its slow training: the agent has to learn the whole task at once, from the take-off to the delivery of the payload, and since the bonuses of the later subtasks are only collected once the earlier ones are mastered, it needs a very large number of environment steps to discover the desired behavior. Curriculum learning (CL) [17] addresses this problem: instead of training the agent directly on the full task, a sequence of tasks of increasing difficulty is defined. Training starts with an easy task that the agent can learn quickly, and the task is made progressively harder, with each stage starting from the policy learned in the previous one. Since the agent always builds on the skills acquired in the earlier stages, it reaches the desired behavior considerably faster than when trained on the full task from scratch.

3.2.1 Formulation

Following [17], each task of the curriculum is an MDP of the form (3.1):

$$\mathcal{M}_\ell = (\mathcal{S},\, \mathcal{A},\, \mathcal{P}_\ell,\, r_\ell,\, \rho_{0,\ell},\, \mathcal{S}_{f,\ell})\,, \qquad \ell = 1, \ldots, n. \tag{3.11}$$

The last task $\mathcal{M}_n$ is the target task, i.e., the full transportation task of the flat agent. Since the observation and action spaces are identical in all tasks, the same policy network π_ζ can be used throughout the curriculum. The tasks, however, may differ in all other elements of the MDP. The transition function $\mathcal{P}_\ell$ changes with the ranges from which the mass and the radius of the payload

are sampled, since these change the dynamics of the system. The reward function r_ℓ changes, since the terms related to grasping and carrying the payload are only present once grasping is required. The initial state distribution $\rho_{0,\ell}$ changes through the sampling range of the goal position, and the set of terminal states $\mathcal{S}_{f,\ell}$ changes, since losing the payload terminates the episode only in the target task.

The curriculum applied in this thesis is a task-level sequence curriculum, i.e., an ordered list of tasks

$$C = [\,\mathcal{M}_1, \mathcal{M}_2, \ldots, \mathcal{M}_n\,], \tag{3.12}$$

where the intermediate tasks $\mathcal{M}_1, \ldots, \mathcal{M}_{n-1}$ are simplified versions of the target task $\mathcal{M}_n$. Training starts with task $\mathcal{M}_1$, and the policy parameters ζ are always carried over when the task changes. The current task is selected adaptively, based on the return G_0 in (3.3), averaged over the last N episodes, denoted by $\bar{G}_0$:

$$\mathcal{M}_\ell \to \begin{cases} \mathcal{M}_{\ell+1}, & \text{if } \bar{G}_0 \geq \delta_{\text{adv}} \text{ and } \ell < n, \\ \mathcal{M}_{\ell-1}, & \text{if } \bar{G}_0 \leq \delta_{\text{ret}} \text{ and } \ell > 1, \\ \mathcal{M}_\ell, & \text{otherwise,} \end{cases} \tag{3.13}$$

where δ_{adv} and δ_{ret} are the thresholds for advancing to the next task and retreating to the previous one, respectively. The retreat rule prevents the agent from remaining in a task that turned out to be too difficult, e.g., due to forgetting previously learned skills. Each training environment applies (3.13) to its own last N episodes, see Appendix A.1.

3.2.2 Training with and without CL

To evaluate CL, the training of the flat agent with CL is compared to training directly on the target task. The curriculum of the flat agent consists of $n = 4$ tasks, summarized in Table 3.2. The parameters of the task switching rule (3.13) are $N = 100$ episodes, $\delta_{\text{adv}} = 1000$, and $\delta_{\text{ret}} = -1000$.

Task	Description	Grasping	m_{payload} [kg]	R_{payload} [m]	$x_{\text{goal}}, y_{\text{goal}}$ [m]
$\mathcal{M}_1$	Waypoint following	no	–	–	$[-1.5, 1.5]$
$\mathcal{M}_2$	Grasping a nearly fixed payload	yes	0.05	[0.02, 0.025]	$[-1, 1]$
$\mathcal{M}_3$	Grasping a randomized payload	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1, 1]$
$\mathcal{M}_4$	Full task	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1.5, 1.5]$

Table 3.2: Tasks of the curriculum of the flat agent.

In tasks $\mathcal{M}_1$–$\mathcal{M}_3$, the approach waypoint above the payload is omitted, and losing the payload does not terminate the episode. Both are introduced only in the target task $\mathcal{M}_4$.

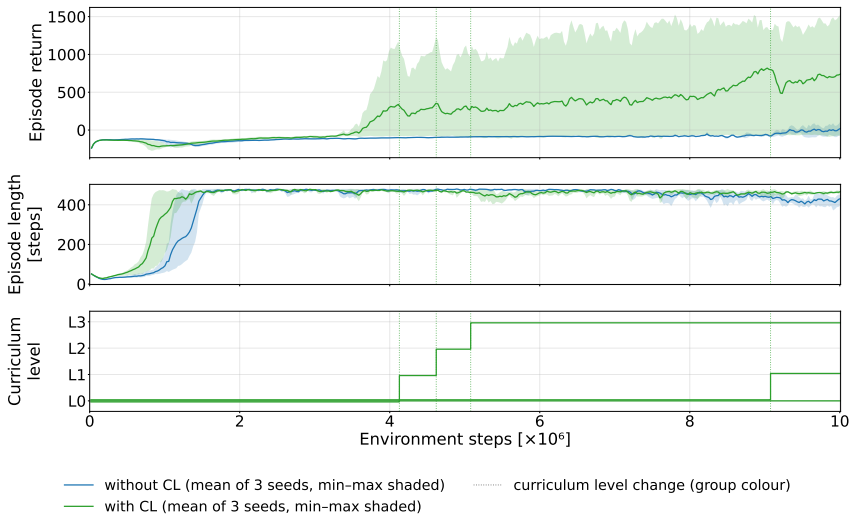


Figure 3.2: Training of the flat agent without CL (blue) and with CL (green): training return (top), episode length (middle), and curriculum task (bottom). Solid lines: mean of three random seeds; shaded areas: min–max range over the seeds; dotted vertical lines: task switches. The levels L0–L3 correspond to the tasks $\mathcal{M}_1$–$\mathcal{M}_4$.

As can be seen in Figure 3.2, the flat agent trained with CL reaches a high episodic return considerably faster than the agent trained without it. However, the learning process shows a high variance across the random seeds: while on some seeds the agent converges quickly, on others the training does not converge within the given number of time steps. Thus, the benefit of CL, although clearly visible on average, is not guaranteed for every individual training run. During training, the agent only ever advanced to the next task, i.e., the retreat rule in (3.13) was never triggered. Since CL improved the learning speed in this example, all policies in the remainder of this thesis are trained with CL.

3.3 Hierarchical Reinforcement Learning

Although CL speeds up learning, the fact that the flat agent commands the motors of the drone directly leads to two problems. First, the agent has to learn not only how to solve the task but also how to stabilize the drone, which slows down learning. Second, the learned policy may not transfer well to the real system, since the real rotors may not follow commands with such a high bandwidth, and unmodeled phenomena, e.g., motor dynamics and control delays, may cause the motors to track the commanded revolutions per minute (RPMs) inaccurately.

To address these issues, a hierarchical reinforcement learning (HRL) approach [12] is proposed, in which two agents work together: the low-level agent tracks a reference velocity and yaw angle, while the high-level agent plans the motion by providing these references.

3.3.1 Formulation

Following [12], the applied hierarchy is a feudal one: a high-level policy sets subgoals for a low-level policy, which in turn determines how these subgoals are achieved. In this thesis, the subgoal is a reference velocity and a reference yaw angle of the drone. Both levels are formulated as in Section 3.1, and each of them has its own observation space, action space, and policy. In the following, the quantities of the high-level and the low-level policy are marked with the superscripts H and L, respectively.

The high-level policy, similarly to the flat agent, solves the transportation task: based on its observation, it selects the references and the tendon commands. It is a parameterized policy with parameters ζ,

$$\pi_\zeta^{\mathrm{H}} : \mathcal{O}^{\mathrm{H}} \to \mathrm{Dist}\big(\mathcal{A}^{\mathrm{H}}\big), \tag{3.14}$$

from which the high-level action is sampled as $A_t^{\mathrm{H}} \sim \pi_\zeta^{\mathrm{H}}(\cdot \mid O_t^{\mathrm{H}})$. The low-level policy computes the motor commands based on its own observation, which contains the references given by the high level. It is a parameterized policy with parameters ν,

$$\pi_\nu^{\mathrm{L}} : \mathcal{O}^{\mathrm{L}} \to \mathrm{Dist}\big(\mathcal{A}^{\mathrm{L}}\big), \tag{3.15}$$

from which the low-level action is sampled as $A_t^{\mathrm{L}} \sim \pi_\nu^{\mathrm{L}}(\cdot \mid O_t^{\mathrm{L}})$. The interaction of the two policies with each other and with the environment is shown in Figure 3.3.

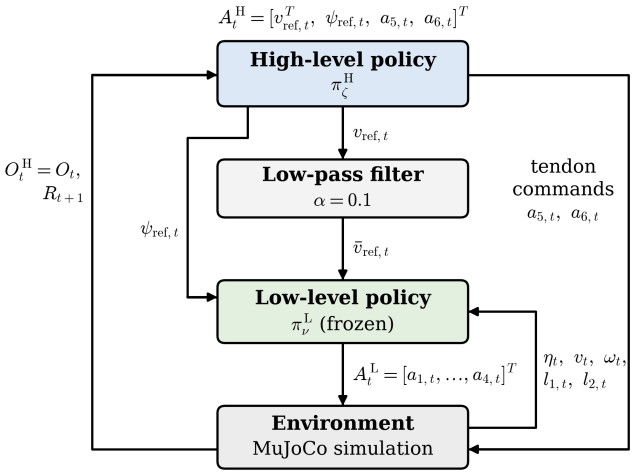


Figure 3.3: Structure of the hierarchical agent. The high-level policy π_ζ^{H} receives the same observation and reward as the flat agent, and provides the reference velocity and yaw angle to the frozen low-level policy π_ν^{L}, as well as the tendon commands to the environment. The reference velocity is smoothed by the low-pass filter (3.20). The low-level policy computes the motor commands based on the state of the drone and the references.

The two policies are trained separately. First, the low-level policy is trained on a velocity tracking task, and then its parameters ν are frozen. Afterwards, the high-level policy is trained on the transportation task, while the frozen low-level policy executes its references. Since the low-level policy handles the stabilization of the drone, the high-level policy only has to learn where the drone should move and how far the manipulator should close; when and toward which side it closes is determined by the environment, see Section 3.1.2. Moreover, as the interface between the two levels is a velocity and yaw reference, the high-level policy can be replaced by any other planner that provides such references, e.g., the model predictive controller presented in the *Model Predictive Control* chapter.

The following subsections define the low-level policy in the same structure as the flat agent.

3.3.2 Low-Level Observation and Action

At time step t, the observation $O_t^{\mathrm{L}} \in \mathbb{R}^{15}$ of the low-level policy is defined as

$$O_t^{\mathrm{L}} = \big[v_t^T,\ \eta_t^T,\ \omega_t^T,\ v_{\mathrm{ref},t}^T,\ \psi_{\mathrm{ref},t},\ l_{1,t},\ l_{2,t}\big]^T, \tag{3.16}$$

where $v_t \in \mathbb{R}^3$, $\eta_t = [\phi_t, \theta_t, \psi_t]^T$, and $\omega_t \in \mathbb{R}^3$ are the linear velocity, the roll, pitch, and yaw angles, and the body-frame angular velocity of the drone, as in the observation (3.5) of the flat agent, $v_{\mathrm{ref},t} \in \mathbb{R}^3$ and $\psi_{\mathrm{ref},t}$ are the reference velocity and the reference yaw angle, and $l_{1,t}$, $l_{2,t}$ are the normalized lengths of the two tendons. The position of the drone is not part of the observation, since the low-level policy only has to track the references, which do not depend on where the drone is.

At time step t, the action $A_t^{\mathrm{L}} = [a_{1,t}, \ldots, a_{4,t}]^T \in [-1, 1]^4$ consists of the four normalized motor commands, as in the action of the flat agent. The tendons are not controlled by the low-level policy, since in the hierarchy, the tendon commands are given by the high-level policy.

3.3.3 Low-Level Reward

Similarly to (3.7), the reward of the low-level policy consists of a dense shaping term, a sparse bonus, and a termination penalty:

$$R_{t+1}^{\mathrm{L}} = r_{\mathrm{shape},t+1}^{\mathrm{L}} + r_{\mathrm{bonus},t+1}^{\mathrm{L}} + r_{\mathrm{term},t+1}^{\mathrm{L}}, \qquad r_{\mathrm{shape},t+1}^{\mathrm{L}} = -e_{v,t+1} - e_{\psi,t+1} - 0.1\,(|\phi_{t+1}| + |\theta_{t+1}|)\,, \tag{3.17}$$

where $e_{v,t+1} = \|v_{t+1} - v_{\mathrm{ref},t}\|$ is the velocity tracking error and $e_{\psi,t+1} = |\psi_{t+1} - \psi_{\mathrm{ref},t}|$ is the yaw error. The shaping term drives the drone toward the reference velocity and yaw angle, and penalizes tilting. The bonus $r_{\mathrm{bonus},t+1}^{\mathrm{L}} = 0.5$ rewards accurate tracking, $e_{v,t+1} < 0.025\,\mathrm{m/s}$, and it is zero otherwise. The termination penalty is $r_{\mathrm{term},t+1}^{\mathrm{L}} = -100$ if the episode is terminated, and zero otherwise.

3.3.4 Low-Level Episode

At the beginning of each episode, the drone is placed level and at rest at $[0, 0, 12.8]^T\,\mathrm{m}$, i.e., high enough above the ground to avoid collisions while tracking the reference; from the second curriculum task on, the initial yaw is random in $[-\pi, \pi)$. During the training of the low-level policy, the references are constant within an episode, i.e., $v_{\mathrm{ref},t} = v_{\mathrm{ref}}$ and $\psi_{\mathrm{ref},t} = \psi_{\mathrm{ref}}$ for all t. The reference yaw angle is set to $\psi_{\mathrm{ref}} = 0$, so that the policy also learns to turn toward the reference. The reference velocity is sampled at the beginning of the episode from a mixture of uniform distributions, so that both hovering and fast motions are sufficiently represented:

$$v_{\mathrm{ref}} \sim \begin{cases} \mathcal{U}\big([-0.1, 0.1]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.4, \\ \mathcal{U}\big([-0.5, 0.5]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.3, \\ \mathcal{U}\big([-1, 1]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.3. \end{cases} \tag{3.18}$$

To make the policy robust to the payload, in about half of the episodes a payload is attached to the drone, already grasped in the fully closed hook, with a random mass and radius, whose ranges are given by the curriculum in Table 3.3; in the other episodes, the tendons are relaxed. An episode is terminated at time step t if the drone flips over, i.e., $|\phi_t| > \pi/2$ or $|\theta_t| > \pi/2$, and truncated after 10 s otherwise.

3.3.5 Low-Level Agent Results

As stated in the *Curriculum Learning* section, the low-level policy is also trained with a curriculum, following the formulation given in (3.11)–(3.13). Its tasks, denoted by $\mathcal{M}_\ell^{\mathrm{L}}$, are summarized in Table 3.3, and the parameters of the task switching rule are $N = 20$ episodes, $\delta_{\mathrm{adv}} = 110$, and $\delta_{\mathrm{ret}} = -1000$.

Task	Description	Random yaw	Payload	m_{payload} [kg]	R_{payload} [m]
$\mathcal{M}_1^{\text{L}}$	Velocity tracking without payload	no	no	–	–
$\mathcal{M}_2^{\text{L}}$	Velocity tracking with a nearly fixed payload	yes	yes	0.05	[0.02, 0.03]
$\mathcal{M}_3^{\text{L}}$	Full task	yes	yes	[0.01, 0.25]	[0.02, 0.04]

Table 3.3: Tasks of the curriculum of the low-level policy.



Figure 3.4: Training of the low-level policy: return (top), episode length (middle), and curriculum task (bottom). Blue: training return; orange and shaded area: mean and min–max range of the evaluation episodes; light red vertical lines: task switches.

As can be seen in Figure 3.4, the training of the low-level policy converges: by the end of the training, the mean training return stabilizes at around 100, while the evaluation return reaches about 150–180. The episode length quickly reaches its maximum of 480 steps, i.e., 10 s at 48 Hz, which means that the drone no longer flips over. The effect of the curriculum is also visible: after the switch to the full task $\mathcal{M}_3^{\text{L}}$, the return drops, since the policy has to learn velocity tracking under more difficult conditions, with payloads of widely varying mass and radius. By the end of the training, however, the policy successfully adapts to these conditions.

To evaluate the trained low-level policy, 1000 test runs were performed with a piecewise constant reference velocity profile of 12 s, in which the reference changes at $t = 1$, 3, 5, 6.5, 8.5, and 10 s. In each run, the initial yaw angle of the drone is random, and in 497 runs a payload with random mass and radius is attached to the drone, while the remaining 503 runs are performed without payload. Unlike in training, the reference changes during the episode, and the episode is longer than 10 s. The results are shown in Figures 3.5–3.7.

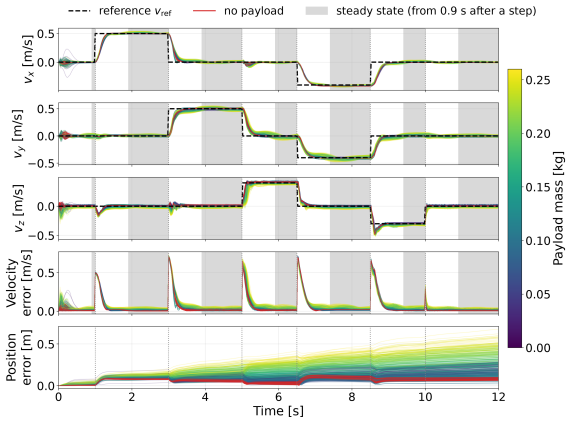


Figure 3.5: Velocity tracking of the low-level policy over 1000 test runs. From top to bottom: velocity components v_x, v_y, v_z with the reference (black dashed), velocity error $\|v - v_{\text{ref}}\|$, and position error $\|p - p_{\text{ref}}\|$ with respect to the integrated reference velocity. Red: runs without payload; other colors: payload mass. Dotted vertical lines: reference steps; gray: steady-state windows, from 0.9 s after each step, in which the steady-state error is evaluated.

As can be seen in Figure 3.5, the low-level policy tracks the reference velocity well in all runs. The response to each reference step is characterized by its overshoot and its settling time, while the tracking accuracy is evaluated in steady state, from 0.9 s after each step, when every run has already settled. In this way, the error is not dominated by the transients, in which the velocity error right after a step equals the size of the step. Although the policy was trained only with constant references, it also tracks a changing reference reliably, which is essential for its use under the high-level policy.

The effect of the payload is most visible in the position error: without payload, it remains below 0.1 m along the 4.2 m long reference path, while for the heaviest payloads it grows up to about 0.5–0.7 m by the end of the run.

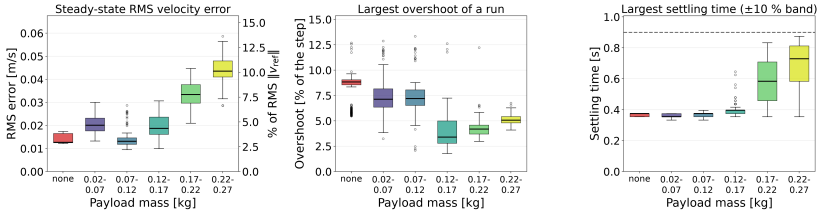


Figure 3.6: Step responses of the low-level policy over 1000 test runs, grouped by the payload mass; "none" denotes the runs without payload. Left: RMS velocity error in the steady-state windows, also in percent of the RMS reference speed of 0.43 m/s in these windows (right axis); middle: largest overshoot of a run above the settled velocity, in percent of the step size; right: largest settling time of a run into a ±10 % band around the settled velocity, dashed: start of the steady-state windows. Boxes: quartiles and median; whiskers: 1.5 times the interquartile range; circles: outliers.

Figure 3.6 quantifies the step responses as a function of the payload mass. Without payload, the steady-state RMS error is 0.014 m/s, i.e., about 3 % of the reference speed. With the payload, the error grows with the mass above about 0.1 kg and reaches 0.04–0.06 m/s, i.e., 10–14 %, for the heaviest payloads, since the policy does not fully compensate for the additional weight. The

overshoot is at most 2 % of the step without payload and at most 7 % with payload, except for the smallest step at 10 s, where it reaches 13 %, i.e., only 0.04 m/s. The settling time is 0.27–0.37 s without payload and grows to at most 0.87 s for the heaviest payloads.

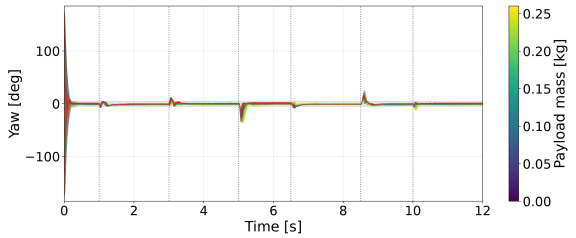


Figure 3.7: Yaw tracking of the low-level policy over 1000 test runs, with a random initial yaw angle and a reference yaw angle of $\psi_{\text{ref}} = 0$. Red: runs without payload; other colors: runs with payload, colored by the payload mass. The gray band indicates $\pm 5°$; dotted vertical lines mark the steps of the reference velocity.

Finally, Figure 3.7 shows the yaw tracking. Starting from a random initial yaw angle in the range of $\pm 180°$, the drone turns to the reference yaw angle within about 0.3 s in all runs, and then keeps the yaw error within $\pm 5°$. The steps of the reference velocity cause short yaw disturbances of up to about 38°, mainly at the vertical steps, where the collective thrust changes abruptly; they are somewhat larger with heavier payloads, but decay within about 0.3 s in every run.

Overall, the evaluation shows that the low-level policy provides fast and robust velocity and yaw tracking, both without and with payloads of various masses. This result is particularly notable considering the range of the payload: its mass varies between 0.02 and 0.26 kg, i.e., up to about 43 % of the mass of the drone, and it hangs below the drone on the compliant manipulator, where its swinging acts as a coupled, nonlinear disturbance. The policy handles this whole range with a single set of parameters, without knowing the mass of the payload, and its tracking error grows only moderately with the mass. With classical control design methods, achieving similar performance would be considerably more difficult: a controller tuned for a single operating point, e.g., a PID controller, would have to be retuned or scheduled for different payload masses, or it would require an explicit estimate of the payload mass and a model of the swinging payload, whereas the learned policy acquires this robustness directly from the randomized training episodes.

3.3.6 High-Level Policy

The high-level policy solves the same transportation task as the flat agent: its observation $O_t^{\text{H}} = O_t$ is given in (3.5), and its reward, initial state distribution, and termination conditions are also the same. It differs from the flat agent only in the action, which is defined at time step t as

$$A_t^{\text{H}} = \left[v_{\text{ref},t}^T,\ \psi_{\text{ref},t},\ a_{5,t},\ a_{6,t}\right]^T, \tag{3.19}$$

i.e., it consists of the reference velocity $v_{\text{ref},t} \in \mathbb{R}^3$ and the reference yaw angle $\psi_{\text{ref},t}$ of the low-level policy (3.16), together with the two tendon commands $a_{5,t}$ and $a_{6,t}$, which are processed in the same way as in the action of the flat agent, see (3.6).

The reference velocity is not passed to the low-level policy directly, but it is smoothed by a first-order low-pass filter:

$$\bar{v}_{\text{ref},t+1} = \bar{v}_{\text{ref},t} + \alpha\left(v_{\text{ref},t} - \bar{v}_{\text{ref},t}\right), \qquad \alpha = 0.1, \tag{3.20}$$

and the low-level policy receives the filtered reference $\bar{v}_{\text{ref},t}$ in place of $v_{\text{ref},t}$ in (3.16). The filter regularizes the action of the high-level policy: abrupt changes of the commanded velocity are smoothed out, so the low-level policy receives a smooth reference, which reduces the oscillations of the drone and of the swinging payload.

3.3.7 High-Level Agent Results

The high-level policy is trained with the same curriculum as the flat agent, given in Table 3.2, so its training can be directly compared to that of the flat agent. Figure 3.8 shows the training of the high-level policy, and Figure 3.9 compares it to the flat agent, both trained on three random seeds.

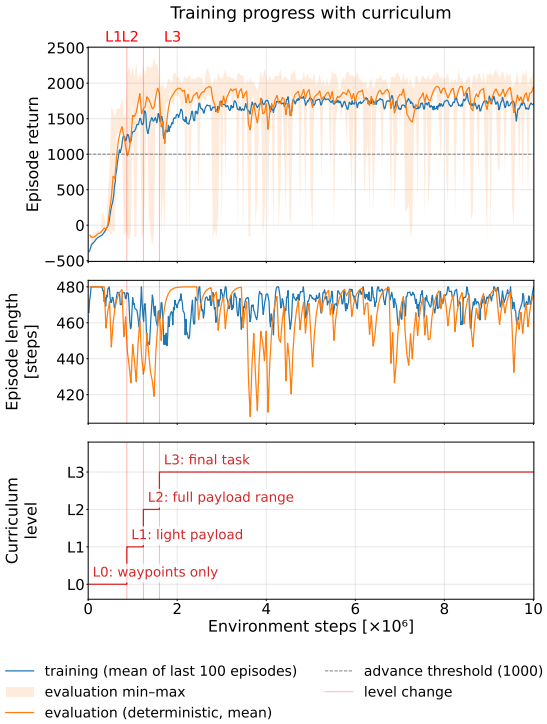


Figure 3.8: Training of the high-level policy: return (top), episode length (middle), and curriculum task (bottom). Blue: training return; orange and shaded area: mean and min–max range of the evaluation episodes; gray dashed line: advancing threshold δ_{adv}; light red vertical lines: task switches.

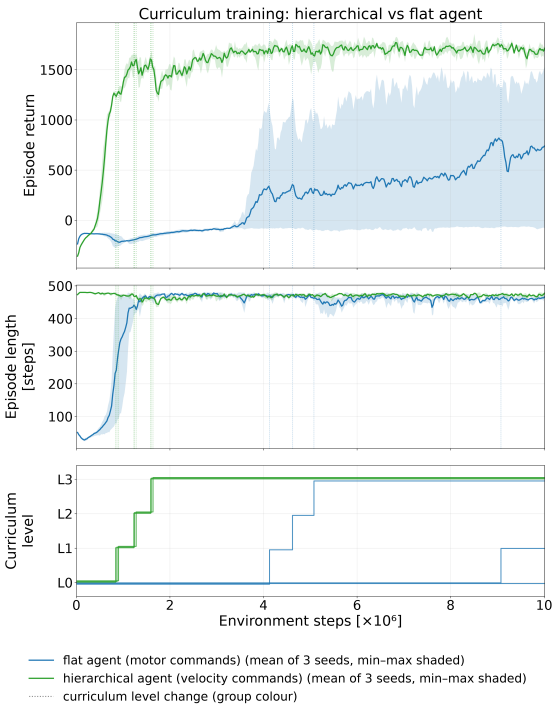


Figure 3.9: Training of the flat agent (blue) and the hierarchical agent (green) with CL: training return (top), episode length (middle), and curriculum task (bottom). Solid lines: mean of three random seeds; shaded areas: min–max range over the seeds; dotted vertical lines: task switches of the individual seeds. In both figures, the levels L0–L3 correspond to the tasks $\mathcal{M}_1$–$\mathcal{M}_4$ in Table 3.2.

As can be seen in Figure 3.8, the training of the high-level policy converges quickly: the agent reaches the target task $\mathcal{M}_4$ after about $1.6 \cdot 10^6$ steps, and after about $3 \cdot 10^6$ steps, the return stabilizes at around 1700 in training and 1700–1900 in evaluation. The episode length stays close to its maximum during the whole training, since the low-level policy keeps the drone stable from the first episode. However, occasional low-return, shorter evaluation episodes remain even at the end of the training, which motivates the detailed validation of the policy.

As can be seen in Figure 3.9, the hierarchical agent learns considerably faster and more reliably than the flat agent. All three of its seeds reach the target task $\mathcal{M}_4$ within about $1.6 \cdot 10^6$ steps, with a narrow spread of the return. In contrast, the flat agent stays in the first task for at least $4 \cdot 10^6$ steps, and by the end of the training, only one of its seeds reaches the target task, while one never leaves the first task, resulting in a mean return of only about 700.

The main reason is that the flat agent first has to learn to stabilize the drone, as shown by its short episodes during the first $1.3 \cdot 10^6$ steps, whereas the hierarchical agent relies on the already trained low-level policy and can spend the whole training on the transport task. Note that this comparison does not include the training steps of the low-level policy. The detailed validation of the high-level policy is presented in the *Comparison* chapter.

Chapter 4

Model Predictive Control

This chapter presents model predictive control (MPC) [20], another method capable of solving the payload transportation task. MPC is widely applied to drone–manipulator systems, for example, for payload transportation with a hook [3], which makes it a natural choice for this work as well.

Compared to the RL-based solution, MPC brings several advantages. First, constraints on the states and the control inputs, such as velocity limits, can be handled explicitly in the optimization problem, which leads to safe and predictable behavior. Second, since MPC relies on a model of the system rather than on previously collected experience, it can also perform well in situations that were not encountered during its design [21]. Third, the smoothness of the control inputs can be enforced directly, e.g., by penalizing their magnitude and their changes in the cost function.

Nevertheless, MPC also has its limitations. Most importantly, it requires an accurate model of the system, and its performance is sensitive to modeling errors. Furthermore, to solve the optimization problem at every control step, the model has to be rolled out over the prediction horizon, and its Jacobian matrices have to be computed, which is computationally demanding, especially on drones with limited onboard resources. The computational demand can be reduced by shortening the prediction horizon, but this may come at the expense of control performance.

Since an accurate analytical model of the drone–manipulator system is difficult to obtain, a data-driven model is applied: a feedforward residual neural network (ResNet) [7] describes the closed-loop behavior of the drone controlled by the low-level policy, i.e., how the drone responds to a given reference velocity. Thus, the MPC replaces the high-level policy of the hierarchical architecture. The first part of this chapter introduces the formulation of the MPC, while the second part presents the identification procedure.

4.1 Model Predictive Control Formulation

MPC is based on numerical optimization: at each control step, the future behavior of the system is predicted over a finite horizon with a model, a cost function is minimized over the future control inputs subject to constraints, and only the first element of the resulting input sequence is applied to the system. At the next control step, the procedure is repeated from the newly measured state, i.e., the horizon is shifted by one step.

The MPC receives the same information about the task as the high-level policy of Section 3.3.6. It runs at the control frequency of 48 Hz, i.e., with the sampling time $\Delta t = 1/48\,\mathrm{s}$, while the reference yaw angle is kept at $\psi_{\mathrm{ref}} = 0$. The MPC does not follow a precomputed reference trajectory: its target is simply the active waypoint of the environment (take-off, approach, grasp, and goal waypoint), which is advanced by the environment in the same way as for the RL agents. The grasping is also handled by the environment: the MPC always requests the fully closed gripper, and the tendons are actuated according to the same rule as for the RL agents, see Section 3.1.2, i.e., only once the payload is close to the gripper and the grasp waypoint is active.

4.1.1 State and Input

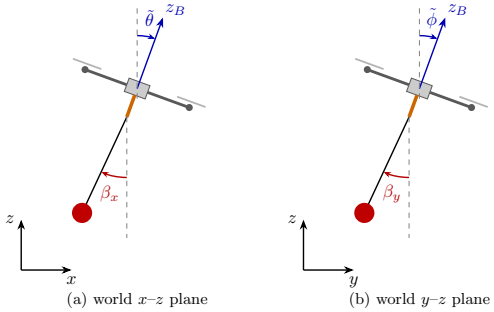
The MPC predicts the motion of the system with its own model, whose state has to be distinguished from the state S_t of the environment introduced in Section 3.1. The state S_t is the full state of the simulation, which also contains, e.g., the configuration of the manipulator and the physical parameters of the payload, and of which the RL agent observes only a part, the observation O_t. The state x of the MPC, on the other hand, is a reduced state of the identified closed loop of the drone and the low-level policy: it contains only the quantities that are needed to predict the motion of this closed loop over the horizon. As required by MPC, this reduced state is assumed to be measured as a whole; in the simulation, all of its components are available. The physical parameters of the payload, however, are not part of x, so, similarly to the RL agent, the MPC does not know the mass and the radius of the payload: their effect is captured by the identified model, as discussed in Section 4.2.3.

The input of the MPC at control step k is the normalized velocity command $u_k \in [-1, 1]^3$, i.e., the reference velocity $v_{\text{ref},t}$ of the low-level policy in (3.20). The index k denotes the control steps, which correspond to the time steps t of the RL formulation. As for the high-level policy, the command is smoothed by the low-pass filter (3.20), so the low-level policy tracks the filtered reference $\bar{v}_{\text{ref},k}$. Since the filter is known exactly, $\bar{v}_{\text{ref},k}$ is included in the state of the model, and its dynamics do not have to be identified.

At control step k, the state of the prediction model is

$$x_k = \left[p_k^T,\ v_k^T,\ \tilde{\eta}_k^T,\ \dot{\tilde{\eta}}_k^T,\ \bar{v}_{\text{ref},k}^T,\ \beta_k^T,\ \dot{\beta}_k^T\right]^T \in \mathbb{R}^{19}, \tag{4.1}$$

where $p_k \in \mathbb{R}^3$ and $v_k \in \mathbb{R}^3$ are the position and the velocity of the drone in the world frame, $\tilde{\eta}_k = [\tilde{\phi}_k,\ \tilde{\theta}_k,\ \psi_k]^T$ is its attitude, $\dot{\tilde{\eta}}_k$ is the time derivative of the attitude, $\bar{v}_{\text{ref},k} \in \mathbb{R}^3$ is the filtered reference velocity (3.20), and $\beta_k = [\beta_{x,k},\ \beta_{y,k}]^T$ and $\dot{\beta}_k$ are the swing angles of the payload and their rates. In the following, the time index is omitted where the quantities are not tied to a specific step, e.g., in the definition of the angles. The state contains every quantity on which the closed loop depends, since the low-level policy observes the attitude, the velocity, the angular velocity, and the filtered reference $\bar{v}_{\text{ref}}$, and the swing angles describe the motion of the attached payload. The tilt angles and the swing angles are illustrated in Figure 4.1.



(a) world x–z plane (b) world y–z plane

Figure 4.1: Tilt angles $\tilde{\theta}$, $\tilde{\phi}$ and swing angles β_x, β_y of the state (4.1). The tilt angles are the angles between the body z_B axis and the world vertical, projected onto the world x–z (a) and y–z (b) planes; the swing angles are the angles between the line from the gripper to the payload and the world vertical, projected onto the same planes.

The tilt angles $\tilde{\phi}$ and $\tilde{\theta}$ are computed from the direction of the body z_B axis, as its angles from the world vertical in the world y–z and x–z planes, respectively, while ψ is the yaw angle; at $\psi = 0$, $\tilde{\phi}$ and $\tilde{\theta}$ coincide with the roll and pitch angles ϕ and θ in η. The tilt angles are defined in the world frame, rather than as the roll and pitch angles of the body, so that they are independent of the heading of the drone and consistent with the world-frame velocity: a motion along the world x axis always corresponds to $\tilde{\theta}$, and a motion along the y axis to $\tilde{\phi}$. The time derivative $\dot{\tilde{\eta}}$, unlike

the body-frame angular velocity ω in (2.4), directly describes the change of these angles. Similarly, the swing angles β_x and β_y are the angles of the line from the gripper to the payload from the world vertical in the world x–z and y–z planes, with $\beta = 0$ when the payload hangs straight down.

4.1.2 Prediction Model

The dynamics of the closed loop change considerably when the payload is lifted. Therefore, two operating modes $\kappa \in \{0, 1\}$ are distinguished: in mode $\kappa = 0$, the drone flies without payload, and the swing states are zero, $\beta_k = \dot{\beta}_k = 0$, while in mode $\kappa = 1$, the payload is attached to the gripper. A separate model is identified for each mode, and the MPC switches between them when the payload is detected to be attached.

In each mode, the dynamics are described by a model F_κ based on a ResNet, which predicts the change of the state over one control step:

$$x_{k+1} = x_k + F_\kappa(x_k,\, u_k)\,. \tag{4.2}$$

The prediction over the horizon is obtained by applying (4.2) recursively, starting from the measured state, i.e., the states are propagated with forward Euler steps based on the ResNet. The structure and the training of F_κ are presented in Section 4.2.

4.1.3 Optimal Control Problem

The target of the MPC at control step k is the active waypoint $w_k \in \mathbb{R}^3$, which is kept constant over the horizon. At the grasp waypoint, the environment actuates the tendons only if the payload is closer to the gripper than at the waypoint itself. Therefore, once the drone is within 3 cm of the grasp waypoint, the target is shifted by the measured offset between the gripper and the payload, so that the gripper ends up 1 cm from the surface of the payload.

With the horizon of K steps, the MPC solves at every control step k an optimal control problem over the predicted states $x_{j|k}$ and commands $u_{j|k}$, where $j = 0, \dots, K$ is the index of the prediction step and k the control step at which the prediction is made:

$$\begin{aligned}
\min_{u_{0|k},\dots,u_{K-1|k}} \quad & \frac{1}{2}\sum_{j=1}^{K-1} \left\|p_{j|k} - w_k\right\|_{Q_p}^2 + \frac{\lambda_K}{2}\left\|p_{K|k} - w_k\right\|_{Q_p}^2 \\
& + \frac{\lambda_u}{2}\sum_{j=0}^{K-1} \left\|u_{j|k}\right\|^2 + \frac{\lambda_{\Delta u}}{2}\sum_{j=0}^{K-1} \left\|u_{j|k} - u_{j-1|k}\right\|^2 && \text{(4.3a)}\\
\text{s.t.} \quad & x_{0|k} = x_k, && \text{(4.3b)}\\
& x_{j+1|k} = x_{j|k} + F_\kappa\big(x_{j|k},\, u_{j|k}\big)\,, \qquad j = 0, \dots, K-1, && \text{(4.3c)}\\
& \left\|u_{j|k}\right\|_\infty \le u_{\max}, \qquad j = 0, \dots, K-1, && \text{(4.3d)}\\
& \left|\tilde{\phi}_{j|k}\right| \le \phi_{\max}, \quad \left|\tilde{\theta}_{j|k}\right| \le \phi_{\max}, \qquad j = 1, \dots, K, && \text{(4.3e)}
\end{aligned}$$

and applies the first command of its solution, $u_k = u^\star_{0|k}$.

In the cost function (4.3a), $\|z\|_Q^2 = z^T Q z$. Its first term penalizes the distance of the predicted position $p_{j|k}$ from the waypoint with the diagonal weight $Q_p = \mathrm{diag}(q_x,\, q_y,\, q_z)$, in which, without the payload ($\kappa = 0$), the horizontal error is weighted much more than the vertical one, so that the drone stays above the grasp pose while it descends onto the payload. With the payload ($\kappa = 1$), the vertical error is weighted the most, so that after the grasp, the height error with respect to the goal waypoint is reduced faster than the horizontal one, which keeps the swinging payload clear of the ground. The second term penalizes the position error at the end of the horizon more strongly, with $\lambda_K > 1$. The third and fourth terms penalize the magnitude of the commands and their change, with $u_{-1|k} = u_{k-1}$ being the command applied in the previous control step; they regularize the problem and make the commands smooth, since with less smooth commands, the closed loop of the low-level policy and the swinging payload tends to oscillate. The initial condition (4.3b) sets the

predicted trajectory to start from the measured state x_k of the system, and (4.3c) is the prediction model (4.2) of the current mode κ. Finally, (4.3d) bounds the velocity commands, and (4.3e) limits the predicted tilt of the drone, which keeps its motion moderate, reduces the swinging of the payload, and keeps the drone in the region where the model is accurate.

Only the position appears in the cost; the other states, such as the velocity, the attitude, and the swing of the payload, are needed only for an accurate prediction of the position. The parameters of the MPC, given in Table 4.1, were tuned on 40 randomly initialized episodes of the transportation task.

Parameter	Symbol	Value
Sampling time	Δt	$1/48\,\mathrm{s}$
Horizon	K	20 steps (0.42 s)
Position weight, $\kappa = 0$	$(q_x,\, q_y,\, q_z)$	(100, 100, 10)
Position weight, $\kappa = 1$	$(q_x,\, q_y,\, q_z)$	(50, 50, 100)
Command weight	λ_u	0.5
Command change weight	$\lambda_{\Delta u}$	30
Terminal weight factor	λ_K	5
Command bound	$u_{\max}$	0.8
Tilt limit	$\phi_{\max}$	15°

Table 4.1: Parameters of the MPC.

Since the prediction model is a neural network, (4.3) is a nonconvex optimization problem, for which only a locally optimal solution can be obtained [8]. It is solved at every control step with a Gauss–Newton sequential quadratic programming (SQP) method, in which the quadratic programs are solved with the DAQP solver [4], warm-started from the shifted solution of the previous step.

4.2 System Identification

As mentioned above, the closed loop of the drone and the low-level policy is identified with a feedforward residual neural network (ResNet), an architecture that has proven to be well suited for system identification [7]. This section presents the structure of the identified model F_κ in (4.2), its training, and its validation.

4.2.1 Model Structure

Since the dynamics do not depend on where the drone is, the input of the network is the state (4.1) without the position, $x'_k \in \mathbb{R}^{16}$, together with the command u_k of Section 4.1.1:

$$\xi_k = \left[x_k'^T,\, u_k^T\right]^T \in \mathbb{R}^{19}. \tag{4.4}$$

The network $\mathcal{N}_\kappa$ of mode κ follows the ResNet architecture of [7], and its connections are shown in Figure 4.2.

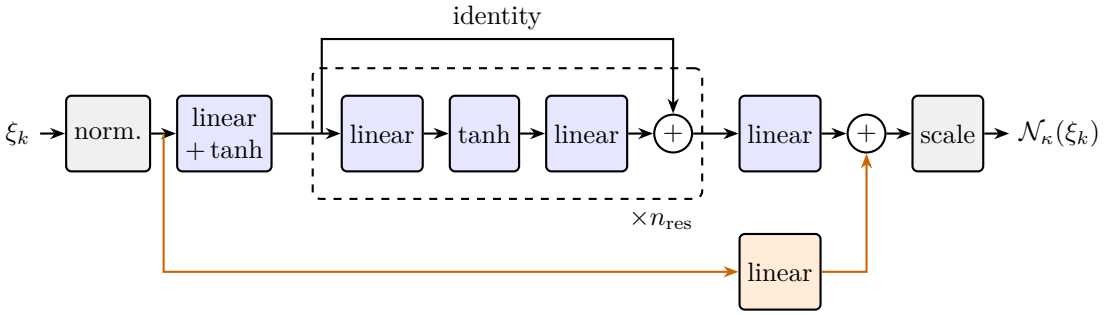


Figure 4.2: Structure of the ResNet $\mathcal{N}_\kappa$, following [7]. The dashed residual block, whose output is the sum of its input and a correction, is repeated n_{res} times, and the linear shortcut (orange) connects the normalized input directly to the output.

Through the identity skip connections, each residual block only adds a correction to its input, which makes the network easy to train, while the linear shortcut represents the linear part of the dynamics directly. The input and the output are normalized with the statistics of the training data, so that all signals have a similar magnitude.

The network predicts the one-step increments of the velocity, the attitude, the attitude rates, and the swing rate:

$$\left[\Delta v_k^T,\ \Delta\tilde{\eta}_k^T,\ \Delta\dot{\tilde{\eta}}_k^T,\ \Delta\dot{\beta}_k^T\right]^T = \mathcal{N}_\kappa(\xi_k) \in \mathbb{R}^{11}. \tag{4.5}$$

From these increments, the state change F_κ in (4.2) is

$$F_\kappa(x_k,\, u_k) = \begin{bmatrix} \Delta t\left(v_k + \frac{1}{2}\Delta v_k\right) \\ \Delta v_k \\ \Delta\tilde{\eta}_k \\ \Delta\dot{\tilde{\eta}}_k \\ \alpha\left(u_k - \bar{v}_{\text{ref},k}\right) \\ \Delta t\left(\dot{\beta}_k + \Delta\dot{\beta}_k\right) \\ \Delta\dot{\beta}_k \end{bmatrix}, \tag{4.6}$$

where the last two rows are zero in mode $\kappa = 0$. That is, the position is integrated with the trapezoidal rule, the filter state follows the known filter (3.20), and the swing angle is integrated from the predicted swing rate. The attitude, on the other hand, is not integrated from its rate, but its increment is predicted directly: the low-level policy makes the roll and yaw angles chatter within a control step, so the sampled attitude rates do not describe the change of the attitude between two samples.

4.2.2 Training

The training data is not generated with purely random commands. Such commands would drive the drone into states that never occur in the transportation task, e.g., large drifts, crashes, or flips, while the payload would rarely be grasped, so hardly any data would be collected in mode $\kappa = 1$. Instead, the data is collected in the transportation task itself, so that the model is trained on states and commands that occur in practice, i.e., in the region where the MPC operates. Two kinds of flights are used for this purpose.

Dataset 1 consists of flights in which a position P-controller flies the waypoints of the task, while random, piecewise-constant velocity commands are added to its output, except while approaching and grasping the payload. In this way, the drone still completes the task and picks up the payload, but the commands and states around its trajectory are excited. 16 such flights are used for training and 1 for validation.

Dataset 2 consists of flights of the trained high-level policy of Section 3.3.6, which provide aggressive commands in the region where the MPC operates. 144 such flights of 15 s are used for training and 4 for validation, with the payload parameters of the target task.

The two datasets complement each other. The flights of the high-level policy in Dataset 2 contain the fast maneuvers and large tilts that occur when the RL agent flies the task, but they explore the commands only along its own, nearly deterministic trajectories. The randomly excited P-controller flights, on the other hand, cover a wide range of commands and states evenly, including the slow and moderate motions that the MPC itself produces. With only the high-level policy's flights, the model would be accurate only near the trajectories of the RL agent. The time steps in which the gripper touches the payload before it is lifted are left out, since the contact forces are not part of the model, and so are the steps of the high-level policy's flights in which the drone is tilted by more than 60°; these are only 62 of about 114 000 steps, the last few steps of 8 flights before the drone flips over. The amount of data was chosen by comparing models trained on 24, 72, and 144 flights of the high-level policy: more flights reduced the prediction error on the fast flights considerably, while training the same model for more epochs did not, since its validation loss already levels off, see below.

A separate network is trained for each mode. Since the MPC relies on multi-step predictions, the network is trained with a multi-step rollout loss: from the measured state x_k, the model (4.2) is applied recursively with the recorded commands $u_k, \dots, u_{k+K_\mathrm{r}-1}$, and the loss is the mean squared error between the predicted and the measured states $x_{k+1}, \dots, x_{k+K_\mathrm{r}}$. The compared states are the velocity, the attitude, and the attitude rates, and in mode $\kappa = 1$ also the swing angles and their rates, each divided by its standard deviation in the training data, so that all of them contribute similarly. Only windows that lie entirely within one mode and contain no left-out steps are used. The rollout horizon K_r grows linearly from 2 to 24 steps (0.5 s) during the first 40 % of the training, so the network first learns accurate one-step predictions and then stable multi-step ones. The weights with the lowest validation loss at the full horizon are kept, and the training parameters are given in Appendix A.1.

The course of the training is shown in Figure 4.3. While the rollout horizon grows, the loss at the full horizon decreases quickly for both modes, since the network learns to predict over longer and longer windows. Once the full horizon is reached at epoch 60, the validation loss levels off, while the training loss keeps decreasing slowly, i.e., the network starts to fit the training flights more closely than it generalizes. Therefore, the weights with the lowest validation loss are kept, found at epoch 67 with the payload and at epoch 106 without it.

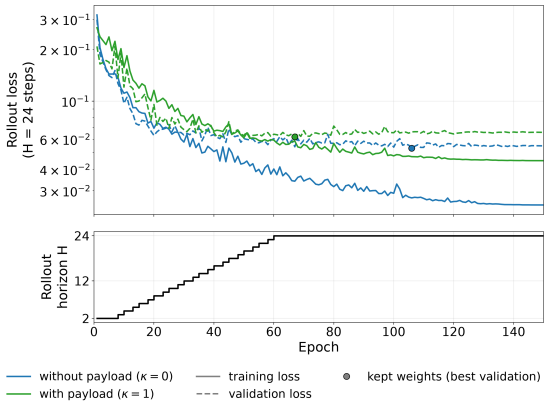


Figure 4.3: Training of the ResNet models. Top: rollout loss over the full horizon of 24 steps on the training and validation data; the dots mark the kept weights. Bottom: rollout horizon K_r used for training.

4.2.3 Validation of the Identification

The identified model is tested on two test sets, which were not used for training or validation. *Test set 1* consists of dedicated test flights. Each test flight consists of 20 s of free flight without payload, in which random, piecewise-constant velocity commands, held for 0.25–1.5 s, are applied around a fixed point, the pick-up of the payload, and 25 s of similarly excited flight with the payload. The payload mass is set to 13 values between 0.01 and 0.25 kg, each flown three times. Neither the MPC nor the high-level policy is involved: from each measured state, the model is rolled out open loop with the applied commands, and its prediction 0.25, 0.5, and 1 s ahead is compared with the simulation. Note that the horizon of the MPC is 0.42 s, so the 1 s prediction is more demanding than what the MPC requires.

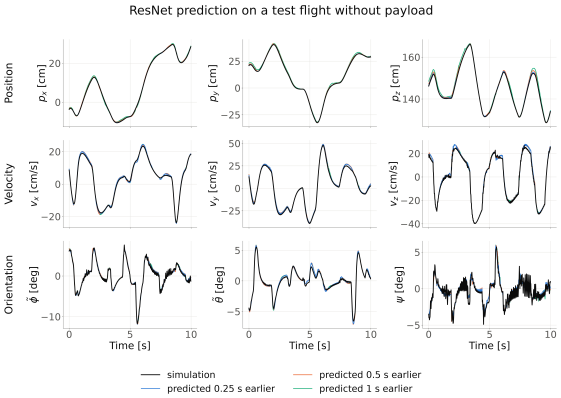


Figure 4.4: ResNet prediction on a test flight without payload: position (top), velocity (middle), and attitude (bottom) over a 10 s stretch. Black: simulation; colored: prediction of the same signal made 0.25, 0.5, and 1 s earlier.

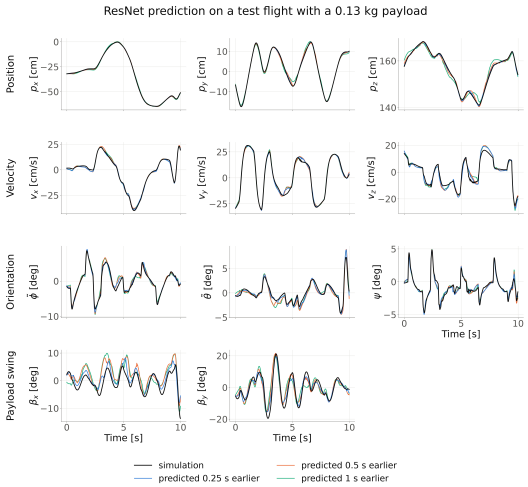


Figure 4.5: ResNet prediction on a test flight with a 0.13 kg payload: position, velocity, attitude, and swing angles of the payload, as in Figure 4.4.

Figure 4.4 shows a test flight without payload. The predictions practically coincide with the simulation even 1 s ahead: over all flights of Test set 1, the RMS prediction error of the individual components is about 1 cm in position, below 1.6 cm/s in velocity, and below 0.7° in attitude, even at the longest look-ahead. Only the high-frequency chatter of the yaw angle, caused by the low-level policy, is not reproduced; the model predicts its mean instead.

With the payload, shown in Figure 4.5, the motion of the drone is still predicted accurately, with RMS errors of the individual components of at most about 2 cm in position, 2.6 cm/s in velocity, and 1° in attitude at 1 s ahead. The swing of the payload is the hardest to predict: its RMS error is about 2° at 0.25 s, and it grows to about 3.7–4.6° at 1 s, as the predicted oscillation accumulates phase and amplitude errors. Since the swing is not penalized in the cost of the MPC, these errors affect the control only through their effect on the motion of the drone.

Figure 4.6 shows how the prediction error depends on the payload mass. The vertical velocity and position errors are the largest for the lightest and the heaviest payloads and the smallest around 0.15 kg. Since the payload mass is not part of the state, the model cannot distinguish between payloads and presumably learns their average effect, which is the most accurate for masses near the middle of the training range. This is a limitation of the model; however, the resulting errors are small compared to the motion of the drone, and since the MPC solves its problem from the measured state at every control step, they are corrected by the feedback. Indeed, the MPC completes the task with payloads of all masses, as shown in the *Comparison* chapter. The swing error is also the largest for the lightest payload.

The absolute errors above have to be judged relative to the motion of the drone and to the accuracy that the task requires. Table 4.2 compares the RMS prediction error at the horizon of the MPC, 0.42 s, with the RMS change of the same quantity during this time, i.e., with the error of predicting no change. Besides Test set 1, the model is also evaluated on *Test set 2*, 24 further flights of the

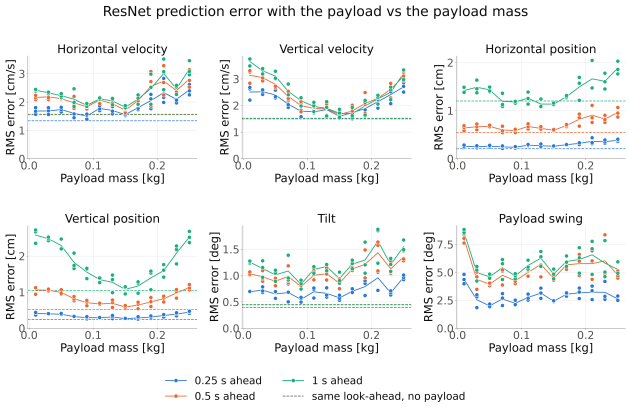


Figure 4.6: RMS prediction error with the payload as a function of the payload mass, 0.25, 0.5, and 1 s ahead. Dots: individual test flights; solid lines: mean over the flights of the same mass; dashed lines: the same error without payload.

high-level policy, recorded in the same way as Dataset 2 but with other random seeds; the safety filter has to predict exactly this fast motion. As the typical values show, Test set 1 is much gentler.

	Test set 1 (test flights)			Test set 2 (high-level policy)		
Quantity	Typical	Change	Error	Typical	Change	Error
Position [cm]	–	12.4	0.9 (7 %)	–	16.8	1.4 (8 %)
Velocity [cm/s]	50	26.0	3.2 (12 %)	116	32.1	7.6 (24 %)
Tilt [°]	8.5	5.7	1.0 (18 %)	13.9	7.4	2.1 (28 %)
Swing [°]	15.3	12.2	4.9 (40 %)	23.9	14.4	5.1 (35 %)

Table 4.2: Prediction error of the ResNet model with the payload, 0.42 s ahead, relative to the motion: typical value (95th percentile of the magnitude), RMS change during 0.42 s, and RMS prediction error, in parentheses in percent of the change. All quantities are vector norms.

Without payload, the errors are 2–8 % of the change in both test sets. With the payload, the position and the velocity errors are 7–12 % in Test set 1 and 8–24 % in Test set 2. In absolute terms, the position error remains below 1.5 cm, an order of magnitude below the waypoint and goal tolerances of 0.1 and 0.15 m, and the tilt error of 1–2.1° is small compared to the tilt limit of 15°. The swing is predicted much less accurately, with an error of 35–40 % of its change. This is not caused by the excluded steps of the training data, which are only 62 steps, but by three other reasons: the swing is a lightly damped oscillation, whose phase errors accumulate; its dynamics depend on the mass and the radius of the payload, which are not part of the state; and large swings are rare in the training data, where 95 % of the swing angles are below 20° and 99 % below 30°. Since the swing enters the MPC only through its effect on the drone, and the swing limit of the safety filter is 45°, these errors are tolerable, but they contribute to the failures that the safety filter cannot prevent with the heaviest payloads. Overall, the identified model predicts the motion of the drone accurately over the horizon of the MPC, both without and with payloads of various masses.

With the identified model, the design of the MPC is complete: the optimal control problem (4.3) uses the ResNet model (4.2) of the current mode for prediction, and its solution provides the velocity commands to the low-level policy in place of the high-level policy. The same identified model is also used by the predictive safety filter presented in the next chapter. The closed-loop performance of the MPC in the transportation task, i.e., its success rate, speed, and computational cost, is evaluated in detail in the *Comparison* chapter, together with the RL agent and the RL agent with the safety filter.

Chapter 5

Safety Filter

As discussed in the *Reinforcement Learning* and *Model Predictive Control* chapters, both methods have their advantages and drawbacks [21]. The RL agent learns the task without an explicit model and computes its commands quickly, but it provides no guarantee that the constraints of the system are satisfied. Moreover, its behavior is reliable only in the region of the state space that was covered during training: in situations for which no training data was available, it may not generalize and may drive the system into an unsafe state. MPC, on the other hand, handles the constraints explicitly, but its performance depends on the accuracy of the model, and at every control step, it requires rolling out the model over the horizon and computing its Jacobian matrices to solve the optimization problem.

This chapter presents a solution that combines the two methods. The challenge it addresses is that the commands of the RL agent may drive the drone into an unsafe state, e.g., into large tilt angles or strong swinging of the payload, from which it cannot recover. Several approaches have been proposed to ensure the safety of learning-based controllers [26]. Control barrier functions provide theoretical safety guarantees, but the safety requirements have to be explicitly encoded as a barrier function, which is not trivial to design for a complex system. Constrained reinforcement learning methods enforce the constraints during the training of the policy, but they are tied to specific RL algorithms and are therefore not modular. Other approaches confine the system to a precomputed safe set of states, e.g., by projecting the commands onto it, but the offline computation of such sets scales poorly with the dimension of the system. Although these approaches are typically computationally cheap during operation, their limitations make them difficult to apply to the drone–manipulator system of this thesis.

A predictive safety filter [31], [26] overcomes these limitations: it can be applied to any, potentially unsafe, controller, independently of how it was trained, and the safe set is defined only implicitly, by the requirement that a safe backup trajectory exists from the next state, which is computed online with an MPC problem. Accordingly, the hierarchical agent is extended with an intermediate step: before a command of the high-level policy is applied to the system, the predictive safety filter checks with the identified model whether the command would drive the drone into an unsafe state. If it would not, the command is applied unchanged; otherwise, the filter replaces it with the closest safe command, obtained by solving an MPC problem.

The filter is minimally invasive: as long as the commands of the RL agent are safe, they are applied unchanged, so the behavior of the learned policy is preserved, and the filter intervenes only when the predicted state would violate the constraints. The combination is also computationally efficient. The policy network is evaluated quickly, and if the proposed command, followed by the previously planned commands, keeps the predicted state within the constraints, a single rollout of the model is sufficient; an optimization problem is solved only otherwise. At the same time, whenever the filter intervenes, the applied command keeps the state predicted by the identified model within the constraints. Note, however, that this is not a formal safety guarantee for the real system, since the model is only an approximation, and the constraints are softened to keep the problem feasible, as described below.

5.1 Formulation

This section formulates the predictive safety filter. First, it is shown how the filter is integrated into the hierarchical agent of Section 3.3, and which signals it receives and produces. Then, the filter is defined as a policy π^{S} that maps the command proposed by the high-level policy to the command applied to the system: it either certifies the proposal or replaces it with the solution of a safety MPC problem. Finally, the constraints that define the safe states are specified, followed by two additional rules for situations that the prediction model does not describe.

5.1.1 Integration into the Hierarchy

The safety filter is placed between the two levels of the hierarchical agent, and neither policy is changed or retrained. The resulting structure is shown in Figure 5.1, which extends the hierarchical agent of Figure 3.3.

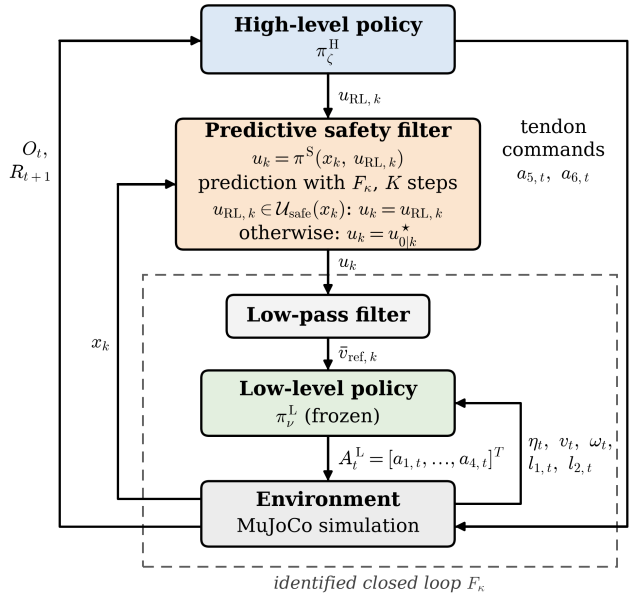


Figure 5.1: Structure of the hierarchical agent with the predictive safety filter π^{S}, following [31]. The filter receives the command $u_{\mathrm{RL},k}$ proposed by the high-level policy and the measured state x_k, and applies either the proposal or the first command $u^\star_{0|k}$ of the safety MPC problem. The dashed frame marks the closed loop described by the identified model F_κ.

At each control step k, which corresponds to the time step t of the RL formulation, the signals of Figure 5.1 are the following. The high-level policy samples its action $A_t^{\mathrm{H}} \sim \pi_\zeta^{\mathrm{H}}(\cdot \mid O_t^{\mathrm{H}})$ from (3.14), based on the observation O_t. Its reference velocity is the proposed command

$$u_{\mathrm{RL},k} = v_{\mathrm{ref},t}, \tag{5.1}$$

while its tendon commands a_5 and a_6 bypass the filter and are applied unchanged.

The safety filter maps the proposal and the measured state x_k of (4.1) to the applied command

$$u_k = \pi^{\mathrm{S}}(x_k,\, u_{\mathrm{RL},k})\,, \tag{5.2}$$

where u_k is the command of Section 4.1.1. To decide on u_k, the filter predicts the state over a horizon of K steps. If the proposal belongs to the set $\mathcal{U}_{\mathrm{safe}}(x_k)$ of safe commands, defined in

Section 5.1.2, it is applied unchanged; otherwise, the filter applies the first command $u^\star_{0|k}$ of the safety MPC problem.

Finally, the applied command is smoothed by the low-pass filter (3.20) in place of $v_{\mathrm{ref},t}$, and the frozen low-level policy (3.15) tracks the filtered reference $\bar{v}_{\mathrm{ref},k}$ with the motor commands $A^{\mathrm{L}}_t \sim \pi^{\mathrm{L}}_\nu(\cdot \mid O^{\mathrm{L}}_t)$, based on its observation (3.16). The low-pass filter, the low-level policy, and the drone, marked by the dashed frame in Figure 5.1, form exactly the closed loop identified in Section 4.2. Therefore, the filter predicts the effect of the commands with the same model as the MPC,

$$x_{j+1|k} = x_{j|k} + F_\kappa\big(x_{j|k},\, u_{j|k}\big)\,, \qquad x_{0|k} = x_k, \tag{5.3}$$

where κ is the current mode, and, as in (4.3), $x_{j|k}$ and $u_{j|k}$ are the predicted state and command at prediction step $j = 0, \ldots, K$, made at control step k.

5.1.2 Safety Filter Problem

The task of the filter is to keep the predicted state within a set of safe states, while changing the proposal as little as possible. The safe states are described by bounds on constrained quantities $y_h(x)$, grouped into constraint groups $h \in \mathcal{G}$:

$$\underline{y}_h \le y_h(x_{j|k}) \le \overline{y}_h, \qquad h \in \mathcal{G},\ j = 1, \ldots, K, \tag{5.4}$$

whose groups and bounds are specified in Section 5.1.3. A command is safe if, after applying it, the drone can still be kept within these bounds over the whole horizon. Accordingly, the set of safe commands in the state x_k is

$$\mathcal{U}_{\mathrm{safe}}(x_k) = \big\{u_{0|k} \ :\ \exists\, u_{1|k}, \ldots, u_{K-1|k} \text{ such that } \|u_{j|k}\|_\infty \le u^{\mathrm{S}}_{\max} \text{ and (5.3), (5.4) hold}\big\}\,. \tag{5.5}$$

The filter policy π^{S} consists of two steps. First, the proposal is certified:

$$\pi^{\mathrm{S}}(x_k,\, u_{\mathrm{RL},k}) = u_{\mathrm{RL},k}, \qquad \text{if } u_{\mathrm{RL},k} \in \mathcal{U}_{\mathrm{safe}}(x_k). \tag{5.6}$$

To check this condition, the proposal is first followed by the shifted plan of the previous step and predicted with (5.3); if this prediction already satisfies (5.4), the proposal is certified without optimization. Otherwise, a feasible continuation $u_{1|k}, \ldots, u_{K-1|k}$ is searched for with $u_{0|k} = u_{\mathrm{RL},k}$ fixed, and the proposal is certified if one is found.

If the proposal cannot be certified, the filter intervenes, and it applies the first command $u^\star_{0|k}$ of the solution of the safety MPC problem

$$\begin{aligned}
\min_{u_{0|k},\ldots,u_{K-1|k}} \quad & \lambda_0 \left\|u_{0|k} - u_{\mathrm{RL},k}\right\|^2 + \lambda_1 \sum_{j=1}^{K-1} \left\|u_{j|k} - u_{\mathrm{RL},k}\right\|^2 \\
& + \lambda^{\mathrm{S}}_{\Delta u} \sum_{j=0}^{K-1} \left\|u_{j|k} - u_{j-1|k}\right\|^2 && \text{(5.7a)} \\
\text{s.t.} \quad & x_{0|k} = x_k, \qquad x_{j+1|k} = x_{j|k} + F_\kappa\big(x_{j|k},\, u_{j|k}\big)\,, \qquad j = 0, \ldots, K-1, && \text{(5.7b)} \\
& \underline{y}_h \le y_h(x_{j|k}) \le \overline{y}_h, \qquad h \in \mathcal{G},\ j = 1, \ldots, K, && \text{(5.7c)} \\
& \left|\tilde{\phi}_{j|k}\right| \le \phi_{\mathrm{hard}}, \quad \left|\tilde{\theta}_{j|k}\right| \le \phi_{\mathrm{hard}}, \qquad j = 1, \ldots, K, && \text{(5.7d)} \\
& \left\|u_{j|k}\right\|_\infty \le u^{\mathrm{S}}_{\max}, \qquad j = 0, \ldots, K-1, && \text{(5.7e)}
\end{aligned}$$

i.e.,

$$\pi^{\mathrm{S}}(x_k,\, u_{\mathrm{RL},k}) = u^\star_{0|k}, \qquad \text{if } u_{\mathrm{RL},k} \notin \mathcal{U}_{\mathrm{safe}}(x_k). \tag{5.8}$$

In the cost function (5.7a), the first term keeps the applied command as close as possible to the proposal, the second term keeps the rest of the plan close to it as well, and the third term, with $u_{-1|k} = u_{k-1}$ being the previously applied command, makes the commands smooth. The weight $\lambda_0 \gg \lambda_1$ makes the intervention as small as possible at the current step, while the rest of the plan

only has to be feasible. The constraint (5.7b) is the prediction model (5.3), and (5.7c) are the constraints (5.4), specified in Section 5.1.3. In addition, (5.7d) imposes a hard bound ϕ_{hard} on the tilt angles of (4.1), and (5.7e) bounds the commands. If the hard tilt bound cannot be satisfied, e.g., because the drone is already tilted beyond it, the problem is solved without (5.7d).

Both the certification and (5.7) are solved in the same way as the MPC problem of Section 4.1.3, with Gauss–Newton SQP iterations on the linearized model. The parameters of the filter are summarized in Table 5.1.

Parameter	Symbol	Value
Horizon	K	20 steps (0.42 s)
Weight of the first command	λ_0	10
Weight of the continuation	λ_1	0.1
Command change weight	$\lambda^{\text{S}}_{\Delta u}$	0.5
Command bound	$u^{\text{S}}_{\max}$	1
Hard tilt bound	ϕ_{hard}	60°
SQP iterations	–	2

Table 5.1: Parameters of the safety filter.

5.1.3 Constraints

The constraints (5.7c) are implemented as soft constraints, i.e., their violation is not forbidden, but it is heavily penalized in the cost, so that the problem remains feasible even if no command can satisfy them exactly, e.g., when the drone is already outside the bounds. The filter uses four constraint groups $\mathcal{G} = \{\text{tilt}, \text{altitude}, \text{speed}, \text{swing}\}$ in (5.4), summarized in Table 5.2: the drone must not tilt too much, approach the ground, fly too fast, or let the payload swing too much. The tilt bound of 15° keeps the drone in the region where the identified model is accurate, while the hard bound $\phi_{\text{hard}} = 60°$ in (5.7d) lies well below the 90° at which the drone flips over. The swing constraint is only used in mode $\kappa = 1$, when the payload is attached. The penalties of the violations set the priorities of the groups, tilt > altitude > speed > swing, so that if not all constraints can be satisfied, the tilt is the last to be violated.

Group h	Constrained quantity $y_h(x)$	Bound
Tilt	$\tilde{\phi},\ \tilde{\theta}$	$\lvert\cdot\rvert \le 15°$
Altitude	p_z	$\ge 0.3\,\text{m}$
Speed	$v_x,\ v_y,\ v_z$	$\lvert\cdot\rvert \le 1\,\text{m/s}$
Swing	$\beta_x,\ \beta_y$ (only in mode $\kappa = 1$)	$\lvert\cdot\rvert \le 45°$

Table 5.2: Constraint groups of the safety filter.

If the measured state already violates a bound, e.g., because the payload already swings more than the limit, an immediate return is impossible. In this case, the bound is relaxed to the current value and has to shrink by a factor of 0.9 per predicted step, so that the state is driven back gradually.

5.1.4 Additional Rules

Two situations are handled by additional rules, since the prediction model does not describe them. First, when the hook touches the payload before it is lifted and the drone starts to tilt by more than 10°, the hook is being levered against the payload, and the drone would flip within about 0.5 s. Since the model contains no contact forces, this is detected from the measured state, and the first command is restricted to a vertical climb until the contact is released and the drone is level again. Second, if the drone stalls, i.e., it moves less than 0.1 m within 1 s while it is farther than 0.25 m from the active waypoint, the MPC of Section 4.1 takes over for 0.5 s and flies toward the waypoint; its commands pass through the same filter.

Chapter 6

Comparison

This chapter compares the three methods that solve the transportation task with the same low-level policy: the RL director, i.e., the high-level policy of Section 3.3.6, the MPC director of the *Model Predictive Control* chapter, and the RL director extended with the safety filter of the *Safety Filter* chapter. The comparison focuses on the question of whether the safety filter is able to save the RL director in the situations in which it fails.

6.1 Evaluation Setup

The methods are evaluated on 500 randomly generated scenarios of the transportation task described in Section 2.3, with a scene similar to Figure 2.5. The task parameters are sampled from the bounds of Table 2.2, which correspond to the final curriculum level. In the following, the payload mass denotes the total mass of the payload, i.e., the disk mass m_{payload} plus the 0.01 kg of the cylinder, which lies between 0.02 and 0.26 kg. Each scenario is defined by a random seed, so all three methods fly exactly the same scenarios. Moreover, the two runs of the RL director with and without the filter differ only through the interventions of the filter. An episode ends when the success condition below is fulfilled, when the drone crashes, or after at most 25 s, and its outcome is one of the following:

- *success*: the drone stays within 0.15 m of the goal with the payload on the hook for 5 s without interruption and without a crash,
- *stability*: the drone flips over, i.e., its roll or pitch angle exceeds 90°, or it hits the ground,
- *payload*: the payload is lifted but lost again, and
- *unfinished*: none of the above happens within the time limit.

6.2 Results

The results are summarized in Figure 6.1. The RL director completes 468 of the 500 scenarios (93.6 %), the MPC director 497 (99.4 %), and the RL director with the safety filter 484 (96.8 %). Almost all failures of the RL director are stability failures after the payload has been lifted. As the success rate as a function of the payload mass shows, the failures concentrate on the heaviest payloads: for the heaviest fifth of the scenarios, above 0.21 kg, the success rate of the RL director drops to 71 %, which the safety filter raises to 86 %, while the MPC director still succeeds in 97 % of these scenarios.

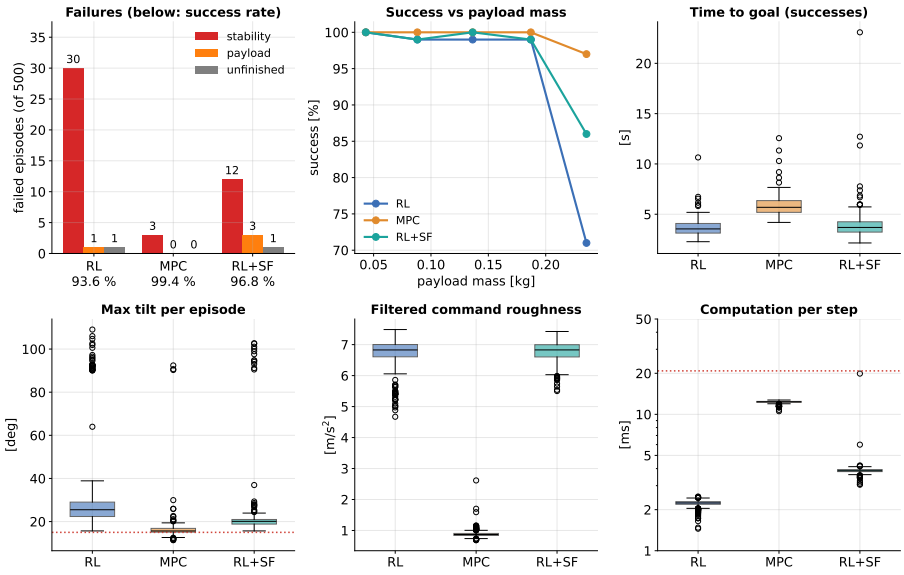


Figure 6.1: The RL director (RL), the MPC director (MPC), and the RL director with the safety filter (RL+SF) on the same 500 scenarios. Dotted lines: tilt limit of 15° and control period of 20.8 ms.

The higher success rate of the MPC comes at the cost of speed: the median time to reach the goal is 3.5 s for the RL director, but 5.7 s for the MPC director, since the tilt limit and the penalty on the command changes keep its motion moderate. The safety filter preserves the speed of the RL director, with a median of 3.7 s, since it intervenes in only 4.3 % of the control steps. The same pattern appears in the maximum tilt of the drone: the median is 25.5° for the RL director, 15.7° for the MPC director, and 20.1° with the safety filter.

The smoothness of the commands is compared through the filtered command roughness, i.e., the mean rate of change of the filtered reference velocity $\bar{v}_{\mathrm{ref},k}$ of (3.20), which all three methods pass and which the low-level policy tracks, see Figure 5.1. The filtered commands of the RL director change rapidly, with a median rate of change of 6.8 m/s^2, since the high-level policy switches its commands nearly between the bounds and relies on the low-pass filter to average them. The filtered commands of the MPC director are about eight times smoother, with a median of 0.9 m/s^2, since their changes are penalized in its cost. The filter keeps the roughness of the RL director, with a median of 6.8 m/s^2, since it changes only a few commands. In terms of computation, the RL director requires a median of 2.2 ms per control step and the safety filter 3.9 ms, since in most steps a single rollout of the model is sufficient to certify the proposal. The MPC director requires 12.3 ms, several times more, since it solves the optimization problem at every control step. Its implementation could still be accelerated considerably, e.g., by generating the solver with the acados software package [29].

6.3 Rescue of the RL Failures

Since all methods fly the same scenarios, the outcome of the RL director can be compared with the outcome of the other two methods scenario by scenario. The MPC director succeeds in 31 of the 32 scenarios in which the RL director fails, and it fails in only two scenarios that the RL director completes. The safety filter turns 23 of the 32 failures of the RL director into a success, while it breaks 7 of its 468 successes: in five scenarios, the drone flips over, and in two, the payload is lost.

Eight of the 9 failures that the filter does not prevent occur with payloads heavier than 0.2 kg, and in these scenarios, the filter is active in 5–71 % of the control steps, with a median of 32 %. Since the filter corrects the proposals only as much as the constraints require, and the identified model is the least accurate with the heavy, strongly swinging payload, this is not always sufficient.

A rescued scenario with a 0.26 kg payload is shown in Figures 6.2 and 6.3. The RL director approaches the goal too fast, overshoots it, and the swinging payload pulls the drone over: 3.8 s after the start, about 1.2 m from the goal, its tilt exceeds 90°. With the safety filter, the filter limits the proposals mainly between 1.4 and 2.3 s, which slows down the drone before it would overshoot, so it reaches the goal after 3.6 s and holds the payload there. The slower MPC director reaches the goal after 5.7 s.

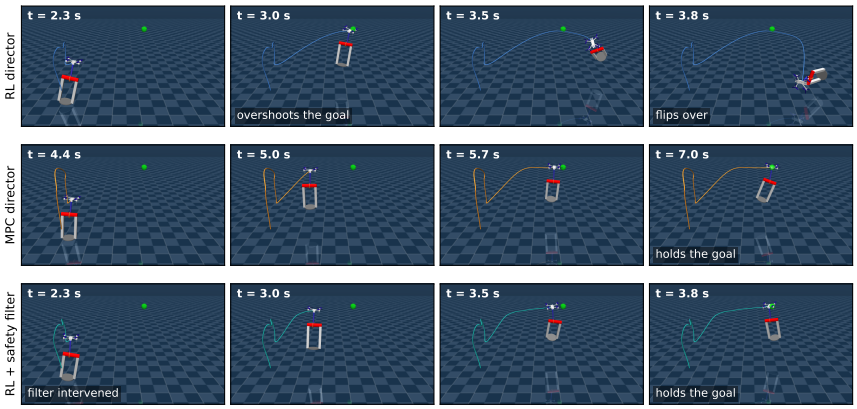


Figure 6.2: Snapshots of the rescued scenario with a 0.26 kg payload, flown by the three methods. The line shows the path of the drone, and the green sphere is the goal. A video of the scenario is available online[1].

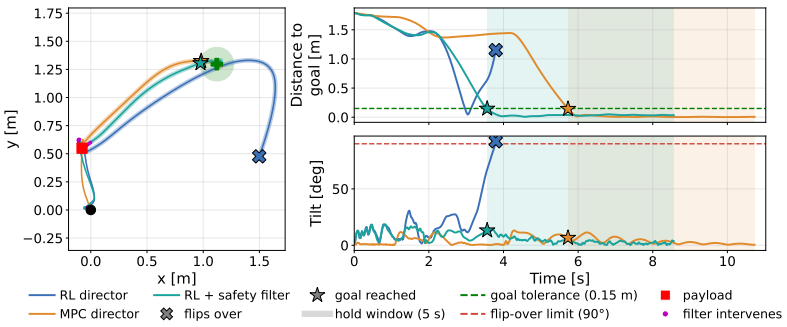


Figure 6.3: The scenario of Figure 6.2 with all three methods: path from above, thick where the payload is carried (left), and the distance to the goal and the tilt (right). Success: within 0.15 m of the goal with the payload (star), kept for 5 s (shaded); stability failure: tilt above 90° (cross).

Overall, the MPC director is the most reliable, but also the slowest and the most computationally demanding method. The RL director is the fastest and requires the least computation, but it fails in about 6 % of the scenarios, mainly with heavy payloads. The safety filter removes almost three quarters of these failures, while it keeps the speed of the RL director at a much lower computational cost than the MPC. Further technical details are given in Appendix A.2.

[1] https://youtu.be/5k_MBPPLFM0

Chapter 7

Conclusion and Future Work

This thesis investigated aerial payload grasping and transportation with a drone equipped with an adaptive hook, a manipulator inspired by spiral robots, which has to grasp a payload with unknown mass and size and transport it to a goal position. In particular, the three research questions formulated in the *Introduction* were investigated.

1. *How can a control method be designed that achieves successful grasping and transportation of a payload with unknown physical parameters with the spiral manipulator, while remaining computationally tractable for real-time operation?*

   It has been shown that both reinforcement learning and model predictive control can be applied to this task. The RL approach, a hierarchical agent trained with curriculum learning, solves the task without a model of the system, while the MPC relies on a ResNet model of the closed loop of the drone and the low-level policy, identified from simulation data. Both methods compute their commands within the control period of 20.8 ms: the RL agent requires a median of 2.2 ms and the MPC 12.3 ms per control step, so both are suitable for real-time operation.

2. *How do reinforcement learning and model predictive control compare in performing the transportation task, in terms of success rate, speed, and computational cost?*

   The two methods were compared on the same 500 randomly generated scenarios, both in terms of safety, i.e., how often the drone flips over or loses the payload, and in terms of performance, i.e., speed and computational cost. The RL agent proved to be the fastest, with a median time of 3.5 s to reach the goal, and the computationally cheapest method, but it completed only 93.6 % of the scenarios and failed mainly with heavy payloads. The MPC was the most reliable, with a success rate of 99.4 %, but its moderate motion made it slower, with a median time of 5.7 s, and it was computationally more demanding.

3. *How can reinforcement learning and model predictive control be combined efficiently to achieve high performance, safety, and computational efficiency in payload transportation?*

   Building on these results, a hybrid method was proposed, in which a predictive safety filter, based on the same identified model, supervises the commands of the RL agent and intervenes only when they would drive the drone into an unsafe state. The filter raised the success rate of the RL agent from 93.6 % to 96.8 %, while it intervened in only 4.3 % of the control steps. Therefore, the speed of the RL agent was preserved, with a median time of 3.7 s to reach the goal, at a computational cost of 3.9 ms per control step, about a third of that of the MPC. This shows that the systematic combination of RL and MPC can unite the speed of the learned policy with the reliability of the model-based approach.

The results of this thesis were obtained in simulation, so the next step is to test the proposed methods on the real system. Practically every component required for this is available in the AIMotionLab of HUN-REN SZTAKI: the Bumblebee quadcopter, whose model is used in this thesis, the motion capture system for measuring the state of the drone and the payload, and the

infrastructure for running the controllers. The transfer to the real system raises several challenges. The policies were trained on the simulated dynamics, so effects that are not modeled, e.g., motor dynamics, delays, and sensor noise, may degrade their performance; this can be mitigated by additional domain randomization during training. Moreover, the positions of the manipulator segments relative to the payload, which are part of the observation of the RL agent, would have to be estimated on the real system, e.g., from the tendon lengths with a kinematic model of the manipulator.

Further improvements are also possible on the methods themselves. The identified model does not know the mass of the payload, which is the main reason for its larger errors with the lightest and heaviest payloads; extending it with an online estimate of the payload mass could improve the predictions of both the MPC and the safety filter, and thus reduce the remaining failures with heavy payloads. Finally, the MPC and the safety filter could be accelerated considerably, e.g., by generating their solvers with the acados software package [29], which would allow longer prediction horizons or their execution on the limited onboard hardware of the drone.

Acknowledgements

This work would never have been completed without the supportive environment that encouraged me throughout it. First of all, I would like to thank my girlfriend, who stood by me and helped me through the more difficult periods, and my family, who supported me without question or hesitation.

I would also like to thank Bence Szinyéri, who provided valuable advice and the framework for this work at the Budapest University of Technology and Economics, and Péter Antal, researcher at HUN-REN SZTAKI, whose insights helped this work move forward.

Bibliography

[1] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019. DOI: 10.1007/s12532-018-0139-4.

[2] Péter Antal, Tamás Péni, and Roland Tóth. Modelling, identification and geometric control of autonomous quadcopters for agile maneuvering. *Repüléstudományi Közlemények*, 35(1): 141–160, 2023. DOI: 10.32560/rk.2023.1.11.

[3] Péter Antal, Andrea Carron, Melanie Zeilinger, Roland Tóth, and Tamás Péni. Robust adaptive predictive control for hook-based aerial transportation between moving platforms. *IEEE Robotics and Automation Letters*, 11(11):12352–12359, 2026. DOI: 10.1109/LRA.2026.3730216.

[4] Daniel Arnström, Alberto Bemporad, and Daniel Axehill. A dual active-set solver for embedded quadratic programming using recursive LDL^T updates. *IEEE Transactions on Automatic Control*, 67(8):4362–4369, 2022. DOI: 10.1109/TAC.2022.3176430.

[5] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. Deep reinforcement learning: A brief survey. *IEEE Signal Processing Magazine*, 34(6):26–38, 2017. DOI: 10.1109/MSP.2017.2743240.

[6] Randal W. Beard and Timothy W. McLain. *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, Princeton, NJ, 2012. ISBN 978-0-691-14921-9.

[7] Pierre Clément Blaud, Philippe Chevrel, Fabien Claveau, Pierrick Haurant, and Anthony Mouraud. Resnet and polynet based identification and (mpc) control of dynamical systems: A promising way. *IEEE Access*, 11:20657–20672, 2023. DOI: 10.1109/ACCESS.2022.3196920.

[8] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[9] Joshua Fishman and Luca Carlone. Control and trajectory optimization for soft aerial manipulation. In *2021 IEEE Aerospace Conference (50100)*, pages 1–17, 2021. DOI: 10.1109/AERO50100.2021.9438502.

[10] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. DOI: 10.1016/S0004-3702(98)00023-X.

[11] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976):982–987, 2023. DOI: 10.1038/s41586-023-06419-4.

[12] Martin Klissarov, Akhil Bagaria, Ziyan Luo, George Konidaris, Doina Precup, and Marlos C. Machado. Discovering temporal structure: An overview of hierarchical reinforcement learning, 2026. URL https://arxiv.org/abs/2506.14045.

[13] William Koch, Renato Mancuso, Richard West, and Azer Bestavros. Reinforcement learning for UAV attitude control. *ACM Transactions on Cyber-Physical Systems*, 3(2):1–21, 2019. DOI: 10.1145/3301273.

[14] Hughes La Vigne, Guillaume Charron, Julien Rachiele-Tremblay, David Rancourt, Ben Nyberg, and Alexis Lussier Desbiens. Collecting critically endangered cliff plants using a drone-based sampling manipulator. *Scientific Reports*, 12(1):14827, 2022. ISSN 2045-2322. DOI: 10.1038/s41598-022-17679-x. URL https://doi.org/10.1038/s41598-022-17679-x.

[15] Chen Liang and Daniel Rakita. Robust and efficient mujoco-based model predictive control via web of affine spaces derivatives, 2026. URL https://arxiv.org/abs/2512.21109.

[16] Ezeldin N. Moustafa, Mitsuhiro Kamezaki, Ahmed Khalifa, Shota Miyake, Tamon Miyake, and Shigeki Sugano. Development of a two-dof three-parallelogram aerial manipulator for pipeline crack repair using a two-tool end-effector and onboard sensor-based localization. In *2025 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1–7, 2025. DOI: 10.1109/AIM64088.2025.11175832.

[17] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *CoRR*, abs/2003.04960, 2020. DOI: 10.48550/ARXIV.2003.04960. URL https://doi.org/10.48550/arXiv.2003.04960.

[18] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[19] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL https://jmlr.org/papers/v22/20-1364.html.

[20] James B. Rawlings, David Q. Mayne, and Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, Madison, WI, 2nd edition, 2017. ISBN 978-0-9759377-3-0.

[21] Rudolf Reiter, Jasper Hoffmann, Dirk Reinhardt, Florian Messerer, Katrin Baumgärtner, Shambhuraj Sawant, Joschka Bödecker, Moritz Diehl, and Sebastien Gros. Synthesis of model predictive control and reinforcement learning: Survey and classification. *Annual Reviews in Control*, 61:101045, 2026. ISSN 1367-5788. DOI: https://doi.org/10.1016/j.arcontrol.2026.101045. URL https://www.sciencedirect.com/science/article/pii/S1367578826000015.

[22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[23] David Silver, Aja Huang, Chris J. Maddison, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. DOI: 10.1038/nature16961.

[24] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 978-0-262-03924-6. URL http://incompleteideas.net/book/the-book-2nd.html.

[25] Manan Tayal. Mujoco-drones-gym: A gpu-accelerated multi-drone simulator for control and reinforcement learning, 2026. URL https://arxiv.org/abs/2606.08039.

[26] Ben Tearle, Kim P. Wabersich, Andrea Carron, and Melanie N. Zeilinger. A predictive safety filter for learning-based racing control. *IEEE Robotics and Automation Letters*, 6(4):7635–7642, 2021. DOI: 10.1109/LRA.2021.3097073.

[27] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. DOI: 10.1109/IROS.2012.6386109.

[28] Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Kallinteris Andreas, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, J Terry, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc., 2025. DOI: 10.52202/085713-4916. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/d7ff1795e8527f6443371c3933bdb52b-Paper-Datasets_and_Benchmarks_Track.pdf.

[29] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.

[30] B. V. Vidyadhara, Lima Agnel Tony, Mohitvishnu S. Gadde, Shuvrangshu Jana, V. P. Varun, Aashay Anil Bhise, Suresh Sundaram, and Debasish Ghose. Design and integration of a drone based passive manipulator for capturing flying targets. *Robotica*, 40(7):2349–2364, 2022. DOI: 10.1017/S0263574721001673.

[31] Kim Peter Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021. ISSN 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2021.109597. URL https://www.sciencedirect.com/science/article/pii/S0005109821001175.

[32] Zhanchi Wang, Nikolaos M. Freris, and Xi Wei. Spirobs: Logarithmic spiral-shaped robots for versatile grasping across scales. *Device*, 3(4):100646, 2025. ISSN 2666-9986. DOI: https://doi.org/10.1016/j.device.2024.100646. URL https://www.sciencedirect.com/science/article/pii/S2666998624006033.

[33] Chao Zeng, Zhan Li, Yipeng Yang, Cunjia Liu, and Chenguang Yang. Learning-based methods for aerial manipulation: A focused review. *IEEE/ASME Transactions on Mechatronics*, 31 (2):2270–2287, 2026. DOI: 10.1109/TMECH.2025.3625915.

[34] Wen-Shuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020. URL https://api.semanticscholar.org/CorpusID:221971078.

Appendix

A.1 Hyperparameters

All policies of the *Reinforcement Learning* chapter are trained with the PPO implementation of Stable-Baselines3 [19], with the hyperparameters of Table A.1. The experience is collected by 8 parallel training environments, and a separate evaluation environment is used to monitor the training. In the training figures, the training return is the mean return of the last 100 completed training episodes, while every $4 \cdot 10^4$ environment steps, i.e., 5000 steps per training environment, the policy is evaluated on 5 episodes in the evaluation environment. In curriculum learning, each of the 8 training environments applies the task switching rule (3.13) to its own last N episodes; the curriculum task shown in the training figures is the highest task reached by the training environments, and the evaluation environment always uses this task. The entropy coefficient is relatively high, which encourages exploration: the training becomes more aggressive, i.e., the return fluctuates more, but it remains manageable. The training parameters of the ResNet models of Section 4.2.2 are given in Table A.2.

Parameter	Value
Actor and critic networks	separate, 2 hidden layers of 64 units, tanh
Learning rate	$3 \cdot 10^{-4}$
Discount factor γ	0.99
GAE parameter λ_{GAE}	0.95
Clip range	0.2
Entropy coefficient	0.01
Value function coefficient	0.5
Steps per environment and update	2048
Minibatch size / epochs per update	64 / 10
Gradient norm clipping	0.5

Table A.1: Hyperparameters of PPO.

Parameter	Value
Hidden units / residual blocks	128 / 2
Final rollout horizon K_{r}	24 steps (0.5 s)
Optimizer	Adam, learning rate 10^{-3}, cosine annealing
Epochs / batch size	150 / 256 windows
Gradient norm clipping	1

Table A.2: Training parameters of the ResNet model.

A.2 Technical Details

The code of this thesis, including the simulation environments, the training of the policies, the identification, the MPC, the safety filter, and the scripts of the figures, is available in the reposi-

tory `https://github.com/Eper00/Adaptive-hook-drone`, which is a fork of the MuJoCo-Drones-Gym simulator [25]. All paths below are relative to the root of this repository.

Software

The simulation is implemented in MuJoCo 3.10 [27], with Gymnasium 1.3 [28] environments, in Python 3.12. The policies are trained with the PPO implementation of Stable-Baselines3 2.9 [19], and the ResNet models with PyTorch 2.13 [18]. In the MPC and the safety filter, the derivatives of the ResNet model are computed with the automatic differentiation of PyTorch, and the quadratic programs of their SQP iterations are solved with the DAQP solver [4] through CasADi 3.8 [1].

Hardware

All computations were carried out on a personal computer with an Intel Core i5-14600KF processor. The policies and the ResNet models were trained on the CPU. In the evaluation of Chapter 6, the episodes ran in parallel processes with one thread each, and the computation times were measured in these processes.

The training times on this setup are summarized in Table A.3. The training of the hierarchical agent as a whole takes about 4.8 h, more than the 2 h of the flat agent; however, as shown in Figure 3.9, the high-level policy reaches the target task within about $1.6 \cdot 10^6$ steps, whereas the flat agent may not reach it at all within 10^7 steps. Moreover, the low-level policy only has to be trained once, and it can be reused with any high-level planner, e.g., with the MPC.

Model	Environment steps	Training time
Flat agent	10^7	2 h
Low-level policy (hierarchical RL)	$5 \cdot 10^6$	50 min
High-level policy (hierarchical RL)	10^7	4 h
ResNet models (identification)	–	20 min

Table A.3: Training times on the hardware described above.

Use of AI Tools

The AI assistant Claude (Anthropic), used through Claude Code, assisted in this work with writing code, i.e., parts of the analysis and plotting scripts listed below and the generation of the figures, and with the preparation of this document, i.e., the formatting of the LaTeX source, the drafting of parts of the text, and its language editing. All results were produced with the author's code and simulations, and all content was reviewed by the author, who takes full responsibility for it.

Results

The trained models and the outputs of the scripts are stored in the `results` directory, summarized in Table A.4. The figures of the thesis are written to `results/thesis_figures`.

Directory	Content
`final/`	final policies: low-level, high-level, and flat agents
`resnet/`	ResNet models, their training data and training log
`seeds/`	training runs on three random seeds
`analysis/velocity/`	test runs of the low-level policy
`analysis/identification/`	validation of the identification
`analysis/methods/`	500 episodes of each method (`episodes.pkl`)
`analysis/scenario/seed_1290/`	the scenario of Figures 6.2 and 6.3
`analysis/seeds/`	training curves of the individual seed groups
`thesis_figures/`	figures of the thesis

Table A.4: Content of the `results` directory.

Scripts

The main scripts and the results they produce are summarized in Table A.5.

Script	Purpose
`utilities/learn.py`	training of the PPO policies
`utilities/train_seeds.sh`	training on three random seeds
`utilities/plot_seeds.py`	Figures 3.2 and 3.9 (`-overlay`)
`utilities/plot_curriculum_training.py`	Figures 3.4 and 3.8
`utilities/test_velocity_policy.py`	Figures 3.5–3.7
`utilities/analyse.py`	single scenarios (`-test scenario`)
`multi_drone_mujoco/envs/resnet_mpc.py`	ResNet model, its training, and the MPC
`utilities/resnet_training_curve.py`	Figure 4.3
`utilities/test_resnet.py`	Figures 4.4–4.6
`utilities/resnet_error_scale.py`	Table 4.2
`multi_drone_mujoco/envs/predictive_safety_filter.py`	predictive safety filter
`utilities/compare_methods.py`	evaluation of the three methods
`utilities/plot_comparison_thesis.py`	Figure 6.1
`utilities/render_rescue_scene.py`	Figure 6.2
`utilities/plot_scenario_thesis.py`	Figure 6.3
`utilities/render_pictures.py`	Figures 1.2, 2.5, and 2.6

Table A.5: Main scripts of the repository.

A.3 Notation

The main symbols of the thesis are summarized in Table A.6, grouped by the chapter in which they are introduced. The time step of the RL formulation is denoted by t and the control step of the MPC and the safety filter by k; the two correspond to each other. The subscript of the time step is omitted where it is not needed.

Table A.6: Notation of the thesis.

Symbol	Meaning
System Modeling	
$\mathcal{F}^i$, $\mathcal{F}^b$	inertial (world) frame and body frame of the drone
p, v	position and linear velocity of the drone in the world frame
$\mathbf{R}$	rotation matrix between the world frame and the body frame
$\eta = [\phi, \theta, \psi]^T$	roll, pitch, and yaw angles of the drone
ω, $\hat{\omega}$	angular velocity in the body frame and its skew-symmetric matrix
$W(\eta)$	Euler angle rate matrix, $\dot{\eta} = W(\eta)\,\omega$

Symbol	Meaning
m, J	mass and inertia matrix of the quadcopter
g, e_3	gravitational acceleration, vertical unit vector $[0,\ 0,\ 1]^T$
Ω_i	angular velocity of propeller $i \in \{1, 2, 3, 4\}$
f_i, $\tau_{z,i}$	thrust force and drag torque of propeller i
c, b	thrust and drag coefficients
f, $\tau = [\tau_x,\ \tau_y,\ \tau_z]^T$	collective thrust and torque vector in the body frame
l_x, l_y	distances of the propellers from the body axes
R_{payload}	radius of the cylinder (handle) of the payload
m_{payload}	mass of the disk of the payload
x_{payload}, y_{payload}, z_{payload}	position of the payload
x_{goal}, y_{goal}, z_{goal}	goal position
ψ_0	initial yaw angle of the drone
Reinforcement Learning	
$\text{Dist}(\mathcal{X})$	set of probability distributions over the set $\mathcal{X}$
$\mathcal{M}$	Markov decision process (MDP)
$\mathcal{S}$, $\mathcal{A}$, $\mathcal{O}$	state, action, and observation spaces
$\mathcal{P}$, $\mathcal{Z}$	transition function and observation function
r, ρ_0, $\mathcal{S}_f$	reward function, initial state distribution, set of terminal states
S_t, A_t, O_t, R_{t+1}	state, action, observation, and reward at time step t
T	last time step of an episode
π_ζ, ζ	policy and its parameters
G_t, γ	return from time step t and discount factor
$\mathcal{J}(\zeta)$	expected return of the policy
w_t	active waypoint
$e_{xy,t}$, $e_{z,t}$	horizontal and vertical position errors from the active waypoint
ε_{wp}	waypoint tolerance
$p_{\text{payload},t}$, $p_{s_j,t}$	position of the payload and of the j-th segment of the manipulator
$d_{j,t}$	relative position of the payload with respect to the j-th segment
$d_{\text{seg},t}$	cumulative segment distance
$l_{1,t}$, $l_{2,t}$	normalized lengths of the two tendons
$a_{1,t}, \ldots, a_{6,t}$	components of the action: motor commands a_1–a_4, tendon commands a_5, a_6
σ_t, μ_t	curling direction and closing magnitude of the manipulator
r_{shape}, r_{bonus}, r_{term}	shaping term, sparse bonuses, and termination penalty of the reward
$\mathcal{M}_\ell$, n	ℓ-th task of the curriculum and number of tasks
C	curriculum, i.e., ordered list of tasks
$\bar{G}_0$, N	return averaged over the last N episodes
δ_{adv}, δ_{ret}	thresholds for advancing and retreating in the curriculum
$(\cdot)^{\text{H}}$, $(\cdot)^{\text{L}}$	quantities of the high-level and the low-level policy
π^{H}_ζ, π^{L}_ν	high-level and low-level policies, with parameters ζ and ν
$v_{\text{ref},t}$, $\psi_{\text{ref},t}$	reference velocity and reference yaw angle
$\bar{v}_{\text{ref},t}$, α	low-pass filtered reference velocity and filter coefficient
$e_{v,t}$, $e_{\psi,t}$	velocity and yaw tracking errors of the low-level policy
Model Predictive Control	
Δt	sampling time
x_k, u_k	state of the prediction model and normalized velocity command
$\tilde{\eta} = [\tilde{\phi},\ \tilde{\theta},\ \psi]^T$	tilt angles in the world frame and yaw angle
$\beta = [\beta_x,\ \beta_y]^T$	swing angles of the payload
κ	operating mode: without (0) or with (1) payload
F_κ	identified model of mode κ, predicting the change of the state
K	prediction horizon
$x_{j\|k}$, $u_{j\|k}$	state and command at prediction step j, predicted at control step k
w_k	target (active waypoint) of the MPC
$Q_p = \text{diag}(q_x,\ q_y,\ q_z)$	position weight
λ_K, λ_u, $\lambda_{\Delta u}$	terminal weight factor, command weight, and command change weight
$u_{\max}$, $\phi_{\max}$	command bound and tilt limit
x'_k, ξ_k	state without the position and input of the network
$\mathcal{N}_\kappa$, n_{res}	ResNet of mode κ and number of its residual blocks
K_{r}	rollout horizon of the training loss
Safety Filter	
π^{S}	safety filter policy

Symbol	Meaning
$u_{\mathrm{RL},k}$	command proposed by the high-level policy
$\mathcal{U}_{\mathrm{safe}}(x_k)$	set of safe commands in the state x_k
$\mathcal{G}$, h	set of constraint groups and a constraint group
y_h, $\underline{y}_h$, $\overline{y}_h$	constrained quantity of group h and its lower and upper bounds
λ_0, λ_1, $\lambda^{\mathrm{S}}_{\Delta u}$	weights of the first command, the continuation, and the command change
$u^{\mathrm{S}}_{\max}$, ϕ_{hard}	command bound and hard tilt bound of the filter